

Enterprise Multi-Tenant SaaS Platform

Complete Build Guide — Authoritative Blueprint v3

Product Vision • BRD • System Architecture • Global Architectural Principles
Module Roadmap • Complete Locked Designs for Modules 1a-17
Cross-Cutting Platform Architecture

Generated July 02, 2026 | Living Document — updated as each module is finalized

This version supersedes all prior exports (v1, v2). No Antigravity implementation prompts are included in this export.
All 17 modules complete. Cross-Cutting Platform Architecture finalized. Three architectural consistency corrections applied.

Table of Contents

- 1. How to Use This Guide
- 2. Setup Instructions
- 3. Phase 1 – Product Vision
- 4. Phase 2 – Business Requirements Document (BRD)
- 5. Phase 3 – System Architecture
- 6. Phase 4 – Module Planning & Build Sequence (Updated Status Table)
- 7. Global Architectural Principles (Permanently Locked)
- 8. Architecture & Dependency Diagrams
- 9. Entity Ownership Summary (Extended through Module 17)
- 10. Module Dependency Matrix (Extended through Module 17)
- 11. Phase 5 – Complete Module Designs (1a–17)
 - a. 1a Authentication
 - b. 1b Organization Management
 - c. 1c RBAC
 - d. 2 Subscription & Billing Engine
 - e. 3 Notification Framework
 - f. 4 Audit Logging
 - g. 5 File Storage
 - h. 6 Employee Management (Core HR)
 - i. 7 Attendance
 - j. 8 Leave Management
 - k. 9 Payroll (MVP Cutoff)
 - l. 10 CRM
 - m. 11 Projects
 - n. 12 Help Desk
 - o. 13 Inventory
 - p. 14 Finance
 - q. 15 Analytics & Reports
 - r. 16 Platform AI Layer
 - s. 17 Marketplace
- 12. Phase 6 – Cross-Cutting Platform Architecture (19 Sections)
- 13. Architectural Consistency Review & Applied Corrections
- 14. Appendix – Working Process Reference
- 15. Glossary
- 16. Decision Log
- 17. Version History
- 18. Future Roadmap Beyond Module 17
- 19. Final Architecture Summary

1. How to Use This Guide

This document is the living build guide for the SaaS platform, now covering all 17 planned modules plus the Cross-Cutting Platform Architecture. It supersedes Blueprint v1 (Module 1a only) and Blueprint v2 (Modules 1a-11). This export represents the point at which every module in the original roadmap has a complete, approved 18-point design, and the architecture has passed an explicit consistency review.

Structure

- Phases 1-4 are strategic/architectural documents, finalized once and reused across every module.
- Section 7 (Global Architectural Principles) applies retroactively and going forward to every module.
- Phase 5 contains one full 18-point business design per module, in build order, for all 17 modules.
- Phase 6 contains the Cross-Cutting Platform Architecture — 19 horizontal domains that apply across every module.
- Rule that never changes: no implementation prompt is generated until all 18 points of a module are finalized and approved, one module at a time, in sequence. No Antigravity prompts are included in this export.

2. Setup Instructions

Accounts & Services

- **Supabase** — PostgreSQL, Auth, and Storage (free tier). RLS enabled on every table, non-negotiable.
- **Google Antigravity** — the AI coding agent that implements each module from its approved prompt (generated separately from this document).
- **GitHub** (or similar) — recommended for commit history across modules; also the CI/CD substrate per Cross-Cutting §12.
- **Node.js (LTS)** — for the Vite/React frontend.
- **Razorpay account** — v1 payment provider (Subscription & Billing Engine), behind a provider-agnostic interface.
- **Resend account** — v1 email provider (Notification Framework), behind a provider-agnostic interface.
- **Self-hosted AI runtime (Ollama or vLLM)** — v1 default implementation philosophy for the Platform AI Layer (Module 16), behind a provider-agnostic interface; cloud providers (Anthropic Claude, OpenAI, Gemini, Groq, OpenRouter) available as interchangeable adapters.

Working Rhythm

- Work one module at a time, in the exact sequence defined in Phase 4.
- Finalize the full 18-point design and get explicit approval before any prompt is generated.
- Do not let Antigravity invent business rules that aren't in the prompt — ambiguity goes back to design, never guessed by the coding agent.

External Dependencies Called Out So Far

Module	Dependency	Decision
Subscription & Billing Engine	Payment gateway	Razorpay (v1 primary), behind a Payment Provider abstraction; Stripe/PayU/PayPal as future adapters
Notification Framework	Email provider	Resend (v1 default), behind a Notification Provider abstraction
Platform AI Layer (Module 16)	AI provider	Provider-agnostic; v1 default philosophy = self-hosted open-source models (Ollama/vLLM); Anthropic Claude, OpenAI, Gemini, Groq, OpenRouter available as interchangeable cloud adapters

3. Phase 1 - Product Vision

Product

A modular, multi-tenant business operating system that lets organizations subscribe to only the modules they need — starting with a tight, production-grade core and expanding into a full suite over time.

Customers

Horizontal, industry-agnostic. Organizations from 10–1,000 employees, initially prioritizing India, the Middle East, and Southeast Asia, with compliance-heavy logic (payroll, tax) built as swappable regional packs rather than hardcoded assumptions.

Differentiation

Enterprise power without enterprise complexity — modern UX, AI-first workflows, fast onboarding, and a clean, API-first, extensible module system.

Strategy

Build like a suite, launch like a wedge. Platform foundation (auth, org management, subscriptions, RBAC, notifications, audit, storage) ships first and is never customer-facing on its own. Business modules layer on top one at a time, each independently maintainable and loosely coupled, each fully designed before implementation begins.

Standard

Every decision assumes a real commercial product at production scale — thousands of tenants, strict tenant isolation, security and auditability by default.

Key Locked Decisions

- **Pricing (v1):** Organization Subscription + Module Subscription only; schema kept extensible for future pricing models, not built now.
- **Workflow automation:** v1 modules get configuration, custom fields, and approval chains only — a generic no-code workflow engine is a deferred future module.
- **Platform services** (Notifications, Audit Logging, File Storage) are internal, reusable capabilities — not customer-subscribable modules. **Platform AI Layer and Marketplace join this category as of Module 16/17's finalization.**

4. Phase 2 – Business Requirements Document (BRD)

1. Tenant & Organization Model

- One user identity may hold membership in multiple organizations; switching is instant, no re-authentication.
- Every organization is a fully isolated tenant — data, membership, roles, permissions, and subscription state never cross tenant boundaries.
- Organizations may define internal structure (branches, departments, teams, business units) as a reusable platform capability.

2. Roles & Access

- Fixed platform roles: Platform Super Admin, Organization Owner, Organization Admin, Application Admin, Manager, Employee.
- Organizations may additionally define custom roles.
- Access control combines custom roles, permission groups, and resource-level permissions. ABAC is a future extension, not v1.

3. Subscription & Onboarding

- Self-service: registration → email verification → automatic tenant provisioning → Organization Owner created → guided onboarding wizard.
- 14-day free trial with full access to all MVP modules.
- Trial expiry → 7-day grace period (read-only access) → lockout (no access, no data deletion).
- Reactivation at any point restores full access immediately with no data loss.

4. Data Ownership & Offboarding

- Organizations own their data at all times.
- Cancellation (explicit Owner action) starts a 30-day retention window; reactivation possible within it; after 30 days, permanent deletion per policy. Audit logs may be retained separately.
- Locked ≠ Cancelled: remaining Locked (billing-driven access suspension) never starts the deletion timer — only an explicit Cancel action does.

5. Module Activation Behavior

- Activating a module immediately surfaces relevant existing platform data.
- Owner/Admin can activate or deactivate modules at any time; deactivation never deletes data; reactivation restores instantly.

6. Non-Functional Requirements

- Target availability 99.9%; responsive, mobile-friendly, fast onboarding.
- Enterprise-grade security and complete auditability; API-first design.
- Multi-language, multi-currency, multi-timezone readiness built in from the start.
- Future compliance roadmap (GDPR, SOC 2, ISO 27001, regional packs) informs architecture now but does not gate MVP delivery.

5. Phase 3 - System Architecture

1. Multi-Tenancy & Data Isolation

Shared database, shared schema, tenant isolation enforced via PostgreSQL Row-Level Security (RLS). Every tenant-owned table carries an `organization_id`; RLS is the single source of truth for isolation, not application code.

2. Identity & Tenant-Context Model

User = platform-wide identity, independent of any organization. Membership = link between User, Organization, and coarse platform Role. One user, many memberships. Active Session Context = which organization the user is currently "inside"; switching swaps context without re-authentication.

3. Platform Services Layer

Authentication, Organization Management, Subscription & Billing, RBAC, Notification Framework, Audit Logging, and File Storage form a platform layer consumed by every business module — never duplicated inside a module. **As of Modules 16-17's finalization, the Platform AI Layer and Marketplace join this layer as shared, non-subscribable platform capabilities.**

4. Core Directory (Shared Platform Data Entities)

Organization structure (Structure Nodes), the Employee record, Contact and Account (Module 10), and Item/Location/Movement (Module 13) are platform-owned, shared entities. Business modules reference these; they never fork their own copy.

5. Subscription Enforcement Architecture

Trial → Grace Period → Locked → Reactivated states are enforced as a centralized cross-cutting gate that every module request passes through, owned exclusively by the Subscription & Billing Engine.

6. Module Registry & Activation Model

A conceptual registry catalogs every installable module and its dependencies on platform/core entities. An org's active module set determines visibility and access — the foundation for the Marketplace (Module 17). **Ownership confirmed during the final architectural consistency review: the Module Registry & Activation Model is owned exclusively by Organization Management** (lifecycle of an organization's enabled modules), with Subscription & Billing Engine continuing to own entitlement/gating, and Marketplace remaining strictly a discovery/catalog layer that invokes — never owns — activation logic.

7. File Storage

Supabase Storage for v1, tenant-scoped via storage-namespacing plus RLS on metadata (defense-in-depth), accessed through an internal Storage Provider abstraction. Files referenced platform-wide by immutable File ID, never raw paths/URLs.

8. Notifications

Channel-agnostic dispatch model via a Notification Provider abstraction. v1 channels: Email (Resend), In-App. Future channels (SMS, WhatsApp, Push) plug into the same contract via new adapters, no dispatch-workflow changes.

9. Security Architecture

- RLS is the primary tenant-isolation boundary; RBAC is a second, finer-grained layer on top — never a replacement for RLS.
- Audit Logging captures all sensitive/state-changing actions platform-wide, append-only, immutable by design.
- External credentials are never assumed — requested explicitly when a module needs them.
- Platform Super Admin access is a deliberate, tightly controlled, always-logged exception to normal tenant isolation.

10. Scalability Considerations

RLS + shared schema scales to thousands of organizations on Supabase without per-tenant infrastructure overhead; a future migration path to database-per-tenant remains open for select enterprise customers.

11. API-First Structure

All module functionality is designed to be reachable through the same conceptual API surface, so the future marketplace vision doesn't require an API rewrite. **Formalized in Cross-Cutting Architecture §1 (API Standards & Versioning).**

6. Phase 4 - Module Planning & Build Sequence

UPDATED PER CORRECTION 1

Modules are grouped into **Platform Foundation** (internal, not customer-subscribable), **Business Modules** (customer-facing, subscribable), and, as of the final consistency review, a distinct **Shared Platform Capability** category for Modules 16-17, which are architecturally platform-layer services (like Notifications/Audit Logging/File Storage) rather than subscribable business modules.

#	Module	Category	Status
1a	Authentication	Platform Foundation	Finalized ✓
1b	Organization Management	Platform Foundation	Finalized ✓ — now also owns the Module Registry & Activation Model
1c	RBAC	Platform Foundation	Finalized ✓
2	Subscription & Billing Engine	Platform Foundation	Finalized ✓
3	Notification Framework	Platform Foundation	Finalized ✓
4	Audit Logging	Platform Foundation	Finalized ✓
5	File Storage	Platform Foundation	Finalized ✓ — Platform Foundation layer complete
6	Employee Management (Core HR)	Business Module (MVP)	Finalized ✓
7	Attendance	Business Module (MVP)	Finalized ✓
8	Leave Management	Business Module (MVP)	Finalized ✓
9	Payroll	Business Module (MVP cutoff)	Finalized ✓ — Launchable suite complete
10	CRM	Business Module (Roadmap)	Finalized ✓
11	Projects	Business Module (Roadmap)	Finalized ✓
12	Help Desk	Business Module (Roadmap)	Finalized ✓
13	Inventory	Business Module (Roadmap)	Finalized ✓
14	Finance	Business Module (Roadmap)	Finalized ✓
15	Analytics & Reports	Business Module (Roadmap)	Finalized ✓
16	Platform AI Layer	Shared Platform Capability (corrected from "AI Assistant / Business Module")	Finalized ✓
17	Marketplace	Platform Discovery Capability (corrected from "Business Module (Roadmap)")	Finalized ✓ — All 17 modules complete

Modules 1a, 1b, 1c were designed jointly as one cohesive identity/tenant/access unit but remain three separate, focused designs (and will produce three separate Antigravity prompts). MVP cutoff is Module 9 (Payroll) — the confirmed launchable suite. Modules 16 and 17 were re-categorized from "Business Module (Roadmap)" to shared platform-layer capabilities during the final architectural consistency review, to match their finalized designs (neither is customer-subscribable; both are consumed platform-wide).

7. Global Architectural Principles (Permanently Locked)

These principles were locked during the Notification Framework design and apply retroactively and going forward to every module in this guide, unless a specific module states an explicit, justified override.

- **Cost Philosophy:** Prioritize free-tier services. Prefer open-source software wherever practical. Prefer self-hostable solutions when they provide long-term advantages. Avoid unnecessary paid services during Version 1.
- **Vendor Independence:** Core business logic must never depend directly on any external provider. Every external service (Authentication, Database, Storage, Email, SMS, Payment, AI, Search, Monitoring, Analytics, etc.) must be accessed through provider abstractions.
- **Free-Tier First Design:** Version 1 assumes a startup operating with minimal infrastructure cost while remaining scalable.
- **Open Standards:** Prefer open protocols, open APIs, standard formats, and open-source libraries wherever practical.
- **Future Flexibility:** Every external dependency should be replaceable without redesigning business modules.

Recurring Platform-Wide Patterns (established module by module, now standard)

- **Never hard-delete business records** — status/lifecycle transitions and archiving only (Organization, Employee, Leave, CRM, Projects, Help Desk, Inventory, Finance all follow this).
- **Centralized interfaces, never parallel logic** — every module routes notifications through the Notification Framework, every sensitive action through Audit Logging, every file operation through File Storage, every access decision through RBAC, and (as of Module 16) every AI capability through the Platform AI Layer. No module reimplements these.
- **Read-only-during-Locked** — the standard pattern (view/read allowed, all writes blocked) applies to Audit Logs, Billing/Subscription/Org info, File Storage, Analytics, and every subscribable Business Module. Payroll and Finance use a stricter variant (no financial-state-changing action at all while Locked).
- **Reference, never duplicate** — every module references shared entities (Employee ID, Structure Node, Contact/Account ID, Item/Location ID) rather than copying their data.
- **Version-aware, never retroactive** — policy/configuration changes (leave policy, attendance policy, salary structure, task/pipeline status sets, inventory policies, prompt templates, dashboard definitions) apply prospectively only; historical records and calculations remain exactly as they were at the time.
- **Immutable IDs everywhere** — every core entity across every module carries a stable, immutable internal ID independent of any mutable display value (name, email, slug, SKU).
- **Fail closed, not open** — unrecognized/unconfigured access checks (RBAC), missing templates (Notifications), and unconfigured AI capability requests (Platform AI Layer) default to denial/safe-failure, never silent success.
- **Never a second source of truth** — Analytics, Marketplace, and the Platform AI Layer are each explicitly designed to read through owning modules' stable interfaces rather than becoming secondary data stores.

8. Architecture & Dependency Diagrams

8.1 High-Level Architecture

CLIENT LAYER — React.js + Vite + Tailwind CSS (Web, Mobile-responsive) — all functionality reachable through one API-first surface			
PLATFORM FOUNDATION LAYER (Modules 1a-5, internal, not subscribable)			
Authentication	Organization Management	RBAC	Subscription & Billing
Notification Framework	Audit Logging	File Storage	
SHARED PLATFORM CAPABILITY LAYER (Modules 16-17, internal, not subscribable)			
Platform AI Layer	Marketplace		
BUSINESS MODULES LAYER (Modules 6-15, subscribable, gated by Billing)			
Employee Mgmt (HR)	Attendance	Leave Management	Payroll
CRM	Projects	Help Desk	Inventory
Finance	Analytics & Reports		

8.2 Module Relationship / Dependency Diagram (Narrative)

Every business module (rows above) consumes, without exception: RBAC (access decisions) • Audit Logging (all sensitive actions) • Notification Framework (all dispatch) • Billing Engine (module-subscription gate + Locked-state enforcement) • File Storage (all attachments) • Platform AI Layer (where AI-assisted features are enabled, via feature flag). Employee ID flows from Employee Management into Attendance, Leave, Payroll, Projects, Help Desk (internal requesters), and Inventory (movement actor). Leave writes "On Leave" status into Attendance. Attendance + Leave finalized data flow into Payroll. Contact/Account (CRM) is referenced by Help Desk, Projects, and Finance. Inventory's movement ledger feeds Finance's valuation/COGS. Payroll, CRM, Projects, Inventory, and Finance all expose stable reporting interfaces consumed read-only by Analytics. Every module's AI-assisted extensibility hook (Help Desk's AI-assisted responses, CRM's future timeline summarization, Analytics' NL querying) routes through the Platform AI Layer, never a direct provider integration. Every module is discoverable/activatable through Marketplace, which itself invokes Organization Management's Module Registry and Billing's gating rather than owning either.

8.3 Shared Entity Diagram (Core Directory)

Core Directory (Shared, Platform-Owned): Structure Node (Organization Management) • Employee (Employee Management) • Contact / Account (CRM, shared) • Item / Location / Movement (Inventory, shared). Referenced (never duplicated) by: Attendance, Leave, Payroll, Projects, Help Desk, Inventory → Employee. Projects, Help Desk, Finance, Marketplace → Contact/Account. Finance → Item/Location/Movement.

9. Entity Ownership Summary

EXTENDED PER CORRECTION 2

Entity	Owning Module	Referenced By	Never Duplicated By
User	Authentication	Every module (via Membership/Employee link)	All
Organization	Organization Management	Every module (organization_id)	All
Membership	Organization Management	RBAC, Billing	All
Structure Node	Organization Management	Employee Management	All
Module Registry & Activation State	Organization Management <i>(confirmed in consistency review)</i>	Billing (gating), Marketplace (discovery/display)	All — Billing owns entitlement/gating only; Marketplace never owns registry data
Custom Role / Permission Group	RBAC	Every module (via Permission Check Interface)	All
Subscription / Organization Status	Subscription & Billing Engine / Org Mgmt (structural status only)	Every subscribable module	All
File / File ID	File Storage	Employee Mgmt, Payroll, CRM, Projects, Help Desk, Inventory, Finance	All
Audit Event	Audit Logging	Every module (write-only, one-way)	All
Employee	Employee Management	Attendance, Leave, Payroll, Projects, Help Desk, Inventory, Finance	All
Attendance Record	Attendance	Payroll (worked-time input)	Leave (writes status only, doesn't own record)
Leave Request / Balance	Leave Management	Attendance (status write), Payroll (finalized-leave input)	All
Payroll Run / Payslip	Payroll	Finance (finalized cost input)	All
Contact / Account	CRM	Projects, Help Desk, Finance, Marketplace (future)	All
Project / Task	Projects	Finance (time-billing input, future)	Help Desk (Ticket is a distinct entity)
Ticket / Comment / Category / Queue	Help Desk	Analytics (reporting interface), Platform AI Layer (future AI-assisted responses)	All — Ticket never merges with Project Task
Item / SKU / Location / Stock Movement	Inventory	Finance (valuation/COGS input), Projects (future material consumption), Marketplace (future product catalog)	All — stock quantity never directly writable by any consuming module
Invoice / Expense Claim / Payment Record / Valuation Entry / Chart of Account / Financial Period	Finance	Analytics (reporting interface)	All — Finance never recalculates Payroll/CRM/Projects/Inventory source data
Dashboard / Report Definition / Widget / Scheduled Report / Report Template	Analytics & Reports	Notification Framework (delivery), Platform AI Layer (future NL querying)	All — Analytics never becomes a secondary source of business data
AI Request / AI Capability / AI Configuration / AI Provider / AI Usage Record	Platform AI Layer	Every business module (via stable capability interface)	All — AI Layer never persists calling-module content, only metering metadata
Marketplace Listing / Extension / Category / Bundle / Review Request / Publication Record	Marketplace	Organization Management (activation invocation), Billing (gating display)	All — Marketplace never owns activation logic or the Dependency Matrix itself

10. Module Dependency Matrix

EXTENDED PER CORRECTION 2

#	Module	Depends On	Consumed By
1a	Authentication	—	All modules
1b	Organization Management	Authentication	RBAC, Billing, Employee Mgmt, all business modules (organization_id); owns Module Registry
1c	RBAC	Org Management, Authentication	All modules (access checks)
2	Subscription & Billing	Org Management	All subscribable modules (gate), Org Management (status display), Marketplace (gating display)
3	Notification Framework	Authentication	All modules (dispatch), Analytics (scheduled delivery)
4	Audit Logging	—	All modules (event ingestion)
5	File Storage	RBAC, Billing, Audit Logging	Employee Mgmt, Payroll, CRM, Projects, Help Desk, Inventory, Finance
6	Employee Management	Org Management, RBAC, File Storage, Billing	Attendance, Leave, Payroll, Projects, Help Desk, Inventory, Finance
7	Attendance	Employee Mgmt, RBAC, Billing	Leave (status target), Payroll (input)
8	Leave Management	Employee Mgmt, Attendance (writes into), RBAC, Billing	Payroll (input)
9	Payroll	Employee Mgmt, Attendance, Leave Mgmt, RBAC, File Storage, Billing	Finance (finalized cost input)
10	CRM	RBAC, File Storage, Billing, Employee Mgmt (ownership ref)	Projects, Help Desk, Finance, Marketplace (future)
11	Projects	RBAC, File Storage, Billing, CRM (optional Account link), Employee Mgmt	Finance (time-billing input, future)
12	Help Desk	CRM (Contact/Account ref), Employee Mgmt (internal requesters/agents), RBAC, File Storage, Notification Framework, Audit Logging, Billing	Analytics (reporting interface), Platform AI Layer (future AI-assisted responses)
13	Inventory	Employee Mgmt (movement actor), File Storage, Notification Framework, Audit Logging, RBAC, Billing	Finance (valuation/COGS input), Projects (future), Marketplace (future catalog)
14	Finance	CRM (Closed Won input), Projects (billable time, future), Payroll (finalized cost), Inventory (movement ledger), Employee Mgmt, RBAC, File Storage, Notification Framework, Audit Logging, Billing	Analytics (reporting interface)
15	Analytics & Reports	Every business module (each exposes its own reporting interface), RBAC, Notification Framework, Audit Logging, Billing, File Storage	Owner/Admin dashboards; future Platform AI Layer NL querying/insights
16	Platform AI Layer	RBAC (capability-level check), Billing (module gate), Audit Logging (metadata-only), Notification Framework (Super Admin alerts)	Help Desk, CRM, Analytics, and any future business module — all via stable capability interface
17	Marketplace	Organization Management (Module Registry — surfaces, never replaces), Subscription & Billing (activation gating, pricing display), RBAC, Audit Logging	Owner/Admin (discovery/activation entry point); future third-party developers (Extension submission, architecture only)

11. Phase 5 - Complete Module Designs (1a-17)

Every module below has completed its full 18-point business design and been explicitly approved. No Antigravity implementation prompts are included in this export — prompts are generated and delivered separately, one module at a time, after this design is finalized.

11.1 Module 1a — Authentication LOCKED

Part of the joint Authentication + Organization Management + RBAC design unit.

1. Purpose

Establish secure, platform-wide user identity — independent of any organization — supporting one account holding memberships across multiple isolated tenants, with session handling that never requires re-authentication when switching organizations.

2. Business Problems Solved

- One person (consultant, multi-branch owner, contractor) uses a single login across multiple client/employer organizations.
- Secure front door every module implicitly trusts, so no module reimplements identity checks.
- Removes onboarding friction via self-service signup with verification.

3. Users Involved

New organization owners signing up • Invited users joining an existing organization • Returning users logging in across multiple orgs • Platform Super Admin (internal, elevated access)

4. User Roles

Standard User — a regular platform identity. Platform Super Admin — internal, tightly-controlled identity type, distinct from org-level roles.

5. Permissions

A user can only initiate password/session actions for their own account. Only Platform Super Admins can perform account-level support actions on another identity — logged via Audit Logging.

6. Features

- Self-service signup (email + password); OAuth noted as future extension.
- Email verification required before full account activation.
- Login with "remember last active organization"; logout; forgot password.
- Account lockout after repeated failed attempts.
- Optional/mandatory MFA (TOTP), org-configurable; always-on for Platform Super Admin.
- Active session list (device/browser/location/last-active) with revoke / "revoke all others."
- New/unrecognized device login email alert.

7. Workflows

- Signup → verify → activate → create org (Owner) or join via invite.
- Login → credentials → MFA if required → session → auto-enter single/last-active org.
- Invited-user signup ties an invite link to a specific org + pending role.
- Password reset invalidates all existing sessions.
- Org switch updates session context, no re-authentication.

8. Edge Cases

- Login before verification — blocked, resend offered.
- Invited email already has an account — resolves to existing identity, never duplicates.
- User removed from an org while active elsewhere — next action fails gracefully, prompts switch.
- Expired/reused tokens rejected clearly.
- Zero-organization user gets a graceful create/join landing state.
- MFA required but not enrolled — routed to mandatory enrollment.

9. Business Rules

- One email = one platform identity, always.
- Unverified email cannot authenticate.
- Platform Super Admin accounts provisioned only via controlled internal process.
- Platform Super Admin MFA is mandatory, non-configurable, no override.
- Session context must always resolve to a valid current membership.

10. Validation Rules

- Email valid format, unique platform-wide.
- Password: min 12 chars, upper/lower/number/special; checked against breach lists where supported.
- Verification/reset tokens time-limited, single-use.
- Password policy centrally configurable, not hardcoded per screen.

11. Security Considerations

- Passwords never stored/logged in plain text (Supabase Auth handles hashing).
- Brute-force protection via throttling/lockout.
- All auth events flow into Audit Logging.
- MFA architected for future additional methods without redesign.
- Anti-enumeration: errors never reveal whether an email exists.

12. Multi-Tenant Considerations

Authentication is deliberately tenant-agnostic — identity exists before and independent of org membership; session context is layered on top.

13. Subscription Considerations

Never gated by subscription state — a user must always be able to log in to see lockout status and reactivate.

14. Notifications

Verification email • Password reset email • New/unrecognized device alert — all via the Notification Framework.

15. Reports

Not applicable directly; login/security events feed Audit Logging.

16. Future Scalability

OAuth/SSO (Google, Microsoft) • Additional MFA methods (SMS, push, hardware key) • Enterprise SSO/SAML.

17. Dependencies

Organization Management (immediately after signup) • RBAC (roles on membership) • Notification Framework (stubbed) • Audit Logging (stubbed).

18. Success Criteria

Signup → verify → login with no support intervention • multi-org switch without re-auth • no duplicate accounts • all auth events auditable • lockout prevents brute force without permanent lockout.

11.2 Module 1b — Organization Management LOCKED

1. Purpose

Establish the Organization as the tenant container itself — its profile, internal structure (Structure Nodes), and Membership records linking platform Users to Organizations. **Also owns the Module Registry & Activation Model (confirmed in the final architectural consistency review).**

2. Business Problems Solved

- Gives every signup a real tenant to land in (Owner created automatically).
- Lets organizations model their real-world structure once, reusable by every future business module.
- Authoritative "which orgs does this user belong to" list for the org switcher.
- Handles membership lifecycle: invite, accept, role assignment, removal, ownership transfer.

3. Users Involved

New Organization Owner (post-signup) • Organization Admin • Invited pending users • Existing members • Platform Super Admin

4. User Roles

Tags a membership with one of five fixed platform roles (Owner, Organization Admin, Application Admin, Manager, Employee). Custom roles/permission logic belong exclusively to RBAC (1c).

5. Permissions

Owner: full control — profile, structure, membership, ownership transfer, deactivation trigger. Organization Admin: manage structure/membership, cannot transfer/delete ownership or the org. Manager/Employee/App Admin: read-only. Platform Super Admin: elevated read/support access, always audit-logged.

6. Features

- Auto org creation at signup (Owner assigned), immutable internal Organization ID + unique editable Organization Slug.
- Org profile management (name, logo, industry, size, locale/timezone/currency).
- Structure Node builder — unified hierarchical entity (type: Branch/Department/Team/Business Unit), unlimited depth.
- Membership management with full lifecycle (Pending Invitation → Active → Suspended → Removed).
- Org switcher data provider for Auth.
- Ownership transfer flow, atomic, audit-logged.
- Organization Status control: Active / Archived only (structural lifecycle).
- **Module Registry & Activation Model:** owns the record of which modules are active for an organization; surfaced to Marketplace for discovery/display and to Billing for gating, but owned exclusively here.

7. Workflows

- Org creation at signup → Owner assigned → onboarding wizard.
- Invite → pending membership → accept → active.
- Ownership transfer: Owner selects an existing Active, verified member → atomic transfer → outgoing Owner auto-demoted to Organization Admin → outgoing Owner blocked from self-removal until complete → audit-logged.
- Structure node deletion: blocked if dependents exist, require reassign-or-archive first.

8. Edge Cases

- Last remaining Owner cannot leave/be removed without transfer.
- Duplicate invite to existing member rejected.
- Ownership transfer to a Pending/Suspended/Removed member blocked.
- Structure node deletion with dependents blocked.
- Slug collision on edit rejected, alternative suggested.
- Org with no structure defined is a valid flat state.

9. Business Rules

- Exactly one Owner per organization, always; transfer is atomic, org never without an Owner even transiently.
- New Owner must already be Active/verified.
- Membership Lifecycle statuses retained historically, never hard-deleted.
- Organization Status (structural, this module only): Active → Archived — never deletes data.
- Subscription lifecycle (Trial/Grace/Locked/Reactivated) exclusively owned by Billing; Org Management reads/displays only, never stores a duplicate copy.
- Structure nodes belong exclusively to one organization.

10. Validation Rules

- Org name required, non-unique platform-wide.

- Org Slug required, unique platform-wide, URL-safe, editable with collision check.
- Structure node names unique within the same parent level.
- Invite email valid format; role must be one of the five fixed platform roles.

11. Security Considerations

- Every org-owned table carries organization_id, RLS-enforced.
- Only Owner/Org Admin mutate membership/structure.
- Every ownership transfer and membership-status change is a mandatory audit-logged event.
- Membership table (crosses tenant boundary) gets special RLS care.

12. Multi-Tenant Considerations

A single User can hold memberships in many orgs; org structure/membership data never leaks across tenants. Never forks/duplicates the Employee record.

13. Subscription Considerations

Organization Management owns only structural status (Active/Archived); reads Billing's subscription-state flag (Trial/Grace/Locked/Reactivated) for display only — never enforces or persists it, avoiding state drift.

14. Notifications

Invite email • role changed • ownership transfer (both parties) • member removed — via Notification Framework.

15. Reports

Basic member/structure directory is core functionality, not a report. Events feed Audit Logging.

16. Future Scalability

Parent-child orgs / franchise-holding models • deeper structure hierarchies • custom org metadata fields • bulk member import (CSV) • organization templates/cloning.

17. Dependencies

Authentication (identity, invite hook) • RBAC (layers custom roles atop the coarse label) • Notification Framework • Audit Logging • Subscription & Billing (read-only status). **Consumed by Marketplace (Module 17) for module discovery/activation surfacing.**

18. Success Criteria

Every signup results in exactly one org, one Owner • invite→accept works end-to-end • ownership never zero/ambiguous • structure never orphans members • org switcher data always accurate and tenant-isolated • Organization Status and Subscription Status never duplicate each other • Module Registry remains the single, unambiguous source of an org's active module set.

11.3 Module 1c — RBAC

LOCKED

1. Purpose

Provide fine-grained, configurable access control on top of coarse platform roles from Organization Management — answering "what can this Membership actually do."

2. Business Problems Solved

- Restricts access beyond the five fixed roles (e.g., a Manager who can approve leave but not view payroll).
- Gives every business module a single, consistent permission-check interface.
- Supports custom roles + permission groups + resource-level permissions, without full ABAC.

3. Users Involved

Owner/Org Admin (define roles, assign permissions) • App Admin/Manager/Employee (subjects) • Every future business module (consumers) • Platform Super Admin (bypass, audit-logged)

4. User Roles

No new identity roles — consumes the coarse Membership role as a starting point and layers custom roles/permission groups on top, scoped per organization.

5. Permissions

Owner: full control • Org Admin: same, except own the Owner's access • App Admin: grantable via custom permission, not default • Manager/Employee: subjects only by default • Platform Super Admin: hardcoded bypass, always audit-logged.

6. Features

- Custom Role creation (org-scoped, unique name).
- Permission Groups (reusable bundles).
- Resource-level permission grants (view/edit/approve on a module/entity type).
- Central Permission Check Interface — `can(user, action, resource)` — every module calls this, none hand-rolls its own.
- A Membership holds one coarse role + zero-or-more custom roles/permission groups.
- Default permission templates seeded per fixed role — functional out-of-box.

7. Workflows

- New org gets seeded default permissions immediately, no configuration required.
- Custom role creation → select grants → save → assignable.
- Runtime check: any module action → central interface → coarse defaults + custom grants → allow/deny.
- Edits take effect within the propagation window (see NFR below); no caching staleness beyond it.
- Role deletion blocked while actively assigned.

8. Edge Cases

- No custom roles assigned → falls back to coarse defaults.
- Conflicting grants → deny wins.
- Deletion attempted while assigned → blocked.
- Coarse role change does NOT auto-remove custom role assignments — reviewed separately.
- Unrecognized resource/action → deny by default (fail closed).
- Platform Super Admin bypass never reachable via any org-level configuration.

9. Business Rules

- Deny-wins on conflicting grants, never resolves toward allow.
- Fail closed on unknown combinations.
- Custom roles/permission groups always organization-scoped.
- A custom role cannot be deleted while assigned to any active Membership — no-empty-drafts rule: at least one permission required before save/assignment.
- Platform Super Admin bypass is hardcoded, never configurable.
- Every business module MUST route access checks through the central interface — no parallel logic permitted.

10. Validation Rules

Custom role/permission group name required, unique within the organization. At least one resource-level grant required to save a non-draft custom role.

11. Security Considerations

- Fail-closed is the primary security posture.
- All role/permission changes audit-logged (audit-log-only for admin-side config changes; users notified only when their own effective permissions change).

- Permission propagation NFR: changes effective platform-wide within a maximum of 60 seconds — architectural requirement, mechanism unspecified.
- RLS remains the primary tenant-isolation boundary; RBAC supplements it, never replaces it.

12. Multi-Tenant Considerations

Custom roles, permission groups, and grants are always organization-scoped — never visible or applicable cross-tenant.

13. Subscription Considerations

Never gated by subscription state — permission checks must always resolve correctly, even for a Grace-Period/Locked organization's read-only access. Module-level access gating remains Billing's separate, additional layer.

14. Notifications

Role assigned/removed from a Membership → notification to the affected user. Admin-side role/permission-group config changes → audit-log only, no real-time notification.

15. Reports

Not built here; feeds Audit Logging for future access-review reporting.

16. Future Scalability

ABAC (explicitly deferred) • time-bound/temporary permission grants • shareable permission templates across sibling orgs • approval-chain-aware permissions.

17. Dependencies

Organization Management (coarse role source) • Authentication (identity) • Audit Logging (stubbed) • Notification Framework (stubbed) • every future business module (consumes the central interface), including the Platform AI Layer's capability-level checks.

18. Success Criteria

Fresh org has working default access with zero configuration • custom roles enforce correctly • no check ever fails open • conflicts always resolve to deny • no cross-org visibility • Super Admin bypass never exposed as configurable.

11.4 Module 2 — Subscription & Billing Engine LOCKED

External dependency: Razorpay (v1 payment provider, behind an abstraction).

1. Purpose

Own and enforce the organization's subscription lifecycle (Trial → Grace Period → Locked → Reactivated) and module-level billing as the single centralized gate.

2. Business Problems Solved

Enforces trial/grace/lockout consistently, without every module reimplementing gating. Supports Organization Subscription + Module Subscription pricing, extensible schema for future models. Single place for Owner/Admin to see status, activate/deactivate modules, manage billing.

3-5. Users, Roles, Permissions

Owner (full billing control, default billing owner) • Org Admin (view-only by default) • Platform Super Admin (elevated support actions, audit-logged). No new identity roles; billing access governed by RBAC.

6. Features

- Subscription state machine: Trial (14 days) → Grace Period (7 days, read-only) → Locked (no access, zero deletion) → Reactivated (instant, full restoration, no data loss).
- Per-module subscription toggles, independently activatable/deactivatable.
- Centralized enforcement gate checked before RBAC/business logic runs on every request.
- Invoice/payment history view; payment method management.
- Payment Provider abstraction — Razorpay v1; Stripe/PayU/PayPal/regional providers addable later as adapters with zero core-logic changes.

7. Workflows

Trial starts automatically at org creation. Trial→Grace (day 14) → Grace→Locked (day 7 of grace) — automatic. Reactivation at any point: payment confirmed → instant full restoration. Module activation/deactivation recalculates billing, no data deletion. Payment failure (recurring) → Grace Period even from Active → retry → resolves to Active or Locked.

8. Edge Cases

Invalid/declined payment on reactivation — clear error, state unchanged. Module deactivated mid-cycle — no refund/proration in v1. Trial-abuse fraud prevention flagged as future. Locked org must still log in and see status. Manual trial/grace extension by Super Admin explicit and audit-logged.

9. Business Rules

Subscription state is the single source of truth, owned exclusively here. Enforcement centralized — no module implements its own check. Locked ≠ Cancelled: no deletion from Locked alone; only explicit Cancel starts the 30-day clock. Auth and billing-status visibility never gated by subscription state. No proration/refunds/coupons/annual/usage-based pricing in v1. Billing events must be idempotent.

10-11. Validation & Security

Payment method validated by gateway itself. Activating a new paid module while Locked is blocked until reactivation. Payment credentials never stored directly — gateway-tokenized references only. Webhooks signature-verified. Every state transition and payment event fully audit-logged.

12-13. Multi-Tenant & Subscription

Subscription state and billing data strictly per-organization, RLS-enforced. This module is the subscription-considerations layer every other module defers to.

14. Notifications

Trial ending soon • Grace period started • Locked • payment success/failure • reactivation confirmed.

15. Reports

Invoice/payment history (feature-level). Deeper analytics deferred to Analytics & Reports.

16. Future Scalability

Per-user/feature/usage-based/add-on/marketplace/enterprise-custom pricing • multiple payment methods • proration • discount codes • annual billing.

17. Dependencies

Organization Management (org-creation events, status display) • Authentication (never gated) • RBAC (billing visibility) • Notification Framework, Audit Logging. **Consumed by Marketplace for activation gating/pricing display.**

18. Success Criteria

Every new org gets a 14-day trial with zero manual setup • state transitions automatic and accurate • reactivation instant, no data loss • no module has its own subscription-check logic • no payment credential stored directly.

11.5 Module 3 — Notification Framework LOCKED

External dependency: Resend (v1 email provider, behind an abstraction).

1. Purpose

Provide a single, channel-agnostic dispatch service every module calls — realizing every prior module's "stubbed notification interface."

2. Business Problems Solved

Realizes the stubs already designed into Auth, Org Management, RBAC, Billing. Prevents vendor lock-in on email/SMS/push providers. Single place for templates, delivery status, future channel expansion.

3-5. Users, Roles, Permissions

Indirect only — Notification Preferences screen and In-App inbox for end users; every other module is a caller; Platform Super Admin has delivery-health visibility.

6. Features

- Channel-agnostic dispatch interface: `notify(user, event_type, payload)` — never a direct SDK call.
- v1 channels: Email, In-App (SMS/WhatsApp/Push future, pure provider adapters).
- Notification Provider abstraction: Resend v1 default; SES/Mailgun/SendGrid/Postmark/SMTP addable later.
- Template system: versionable, localizable-ready, mapped per `event_type`.
- In-App Notification Inbox — durable source of truth, created regardless of email delivery outcome.
- Per-category user preferences; delivery status tracking; retry with backoff.

7. Workflows

Dispatch → resolve template → render → send via channel(s) → record status. Every dispatch also creates an in-app record. Preference check before non-critical sends; security-critical notifications bypass opt-out. Failed sends retried on backoff, then surfaced to Super Admin.

8. Edge Cases

No valid email → fails gracefully, never crashes the calling workflow. Missing template → logged error, never blocks the calling module. Opt-out on security-relevant category ignored. Duplicate dispatch for the same logical event → no duplicate (idempotent via unique event ID). Provider outage → degrades to in-app-only.

9. Business Rules

Security-critical notifications never suppressible by preference. Every dispatched notification always creates an in-app record. No module may bypass this framework. Dispatch calls must be idempotent. Business modules publish events only — the Framework alone renders/dispatches.

10-11. Validation & Security

`event_type` must map to a registered template. Recipient must resolve to a valid, verified platform User. Templates never allow unescaped injection. Delivery logs subject to same RLS/tenant discipline. All dispatch attempts audit-logged.

12-13. Multi-Tenant & Subscription

In-app notifications/preferences per User but org-context-aware, no cross-tenant leakage. Never gated by subscription state — a Locked org's users must still receive billing-related notifications.

14-15. Notifications/Reports

N/A (this module is the layer). Delivery success/failure rates — operational view for Super Admin only.

16. Future Scalability

SMS, WhatsApp, Push • digest/batching • rich in-app actionable notifications.

17. Dependencies

Consumed by every other module (including Analytics' scheduled delivery and the Platform AI Layer's Super Admin operational alerts) • Authentication • Audit Logging.

18. Success Criteria

Every stubbed call resolves with no calling-side changes • security-critical notifications never suppressed • no duplicates from retries • switching email provider requires zero calling-module changes.

Additional Locked Architectural Decisions

Asynchronous processing wherever possible. Templates version-aware, support future localization. Every notification has a unique event identifier guaranteeing idempotency. Failed notifications support configurable retry policies.

11.6 Module 4 — Audit Logging LOCKED

1. Purpose

Provide the single, centralized, append-only record of every sensitive/state-changing action — the real implementation behind every prior module's "structured event emission" stub.

2. Business Problems Solved

Fulfills "complete auditability" NFR. Real, queryable who-did-what-when history for enterprise trust and future SOC 2/ISO 27001/GDPR work. Transparent, reviewable trail for Platform Super Admin actions specifically.

3-5. Users, Roles, Permissions

Every other module (event emitters) • Owner/Admin (own-org viewers) • Platform Super Admin (cross-org, itself logged). No org-level role, including Owner, can modify/delete audit records — write access is exclusively programmatic.

6. Features

- Structured Event Ingestion Interface (actor, action, resource, org context, timestamp, metadata).
- Append-only storage — no update/delete exposed to anyone.
- Org-scoped and platform-wide (Super Admin only) trail views, filterable.
- Retention hooks — audit logs may be retained beyond an org's own data-retention window.
- CSV export in v1.

7. Workflows

Emission (only write path): module action → ingestion interface → persisted immutably → fire-and-forget acknowledgment, never blocking the calling module. Super Admin investigation itself generates a new audit event.

8. Edge Cases

Ingestion failure never blocks the calling module's real operation. Favors completeness over deduplication — duplicate entries from retries acceptable; a dropped entry is not. Each record carries Event ID, Correlation ID, Request ID, timestamp. Org archived/cancelled — trail preserved per Billing's retention rules.

9. Business Rules

Records immutable and append-only, no exception. Every sensitive/state-changing action across every module must emit an event. Ingestion never causes a business operation to fail. Cross-org Super Admin access is itself always logged.

10-11. Validation & Security

Every event requires actor, action type, resource reference, organization_id (or platform-level flag), timestamp, schema version. Append-only enforcement structural, not UI convention. Default storage: same Postgres/Supabase instance for v1, interface designed for future migration.

12-13. Multi-Tenant & Subscription

Every record tagged organization_id (or platform-level flag), RLS-enforced. Audit-log viewing remains available during Grace Period and Locked states — read-only, never gated.

14-15. Notifications/Reports

Not applicable in v1. Foundation for future security/access-review reporting, deferred to Analytics.

16. Future Scalability

Migration path to a dedicated log/analytics store • anomaly detection • full compliance-report generation.

17. Dependencies

Consumed by every other module (including Platform AI Layer's metadata-only events) • RBAC (viewing access).

18. Success Criteria

Every stubbed emission resolves with no calling-side changes • no record ever editable/deletable • org-scoped views leak nothing cross-tenant • ingestion never causes a business operation to fail.

Additional Locked Architectural Decisions

Standardized event schema platform-wide, versioned. Extensible metadata. UTC timestamps only. Provider-independent storage. Internal platform service only — no module ever writes directly except via the centralized interface.

11.7 Module 5 — File Storage LOCKED

Final Platform Foundation module. Backend: Supabase Storage (v1, behind an abstraction).

1. Purpose

Provide a single, tenant-isolated, provider-abstracted file storage service every module uses instead of implementing its own upload/storage logic.

2. Business Problems Solved

Prevents every future business module from reinventing upload/storage/access-control logic. Single place to enforce tenant isolation on files.

3-5. Users, Roles, Permissions

Any authenticated Membership uploading/downloading within their org context • every future business module (consumer) • Platform Super Admin (logged support access). Access to specific files determined by calling-module business logic via RBAC.

6. Features

- Storage Provider abstraction — Supabase Storage v1; R2/S3/MinIO addable later.
- Tenant-scoped file namespacing — isolation enforced at the storage layer itself.
- Single upload/download interface — no direct storage-SDK calls from business logic.
- File metadata (filename, size, MIME, uploader, organization_id, linked resource, timestamp) in Postgres, separate from binary objects.
- Signed/time-limited access URLs only — never permanently public.
- Soft-delete; virus/malware scanning as an unbuilt future hook.

7. Workflows

Upload → validate size/type → store via active provider → persist metadata → return stable File ID. Download → tenant match check → short-lived signed URL. Delete → soft-delete metadata + provider removal.

8. Edge Cases

Upload on behalf of a non-member org rejected regardless of calling-module belief. Signed URL never long-lived. Disallowed type/oversized file rejected before reaching provider. Provider outage fails gracefully. Referencing record deleted doesn't auto-purge file.

9. Business Rules

No module may call a storage SDK directly. Files always tenant-scoped, no cross-org sharing in v1. Never exposed via permanent public URLs. Deletion soft by default.

10-11. Validation & Security

File size/MIME limits centrally configurable. Every upload requires valid organization_id and linked-resource reference. Signed URLs sole access mechanism. Tenant isolation at storage-namespacing layer as defense-in-depth. All actions audit-logged.

12-13. Multi-Tenant & Subscription

Every file strictly isolated per organization_id. Read-only during Locked: existing files viewable/downloadable; new uploads/updates/deletions blocked.

14-15. Notifications/Reports

Not a primary source in v1. Storage-usage/quota reporting deferred to future scalability, owned by Billing.

16. Future Scalability

Storage quota per org (Billing-owned) • virus/malware scanning • cross-org file sharing • provider migration • chunked/resumable uploads.

17. Dependencies

Consumed by every future file-attachment module • RBAC • Audit Logging • Billing.

18. Success Criteria

No file ever accessible cross-tenant • no file ever exposed via permanent public URL • switching provider requires zero calling-module changes • every file action captured in Audit Logging.

Additional Locked Architectural Decisions

Files referenced platform-wide using immutable File IDs, never storage paths/URLs. Metadata and binary objects remain separate responsibilities. Ready for future file versioning/chunked uploads. Signed URLs configurable expiration, never cached permanently.

11.8 Module 6 — Employee Management (Core HR) LOCKED

1. Purpose

Own the authoritative Employee record — the platform-owned "Person" entity that every future HR-adjacent module (Attendance, Leave, Payroll, Help Desk, Inventory, Finance) references rather than forking its own copy.

2. Business Problems Solved

Structured workforce data instead of spreadsheets. Establishes the Employee record once so downstream modules build quickly on top. Cleanly bridges Membership (platform access) and Employee (HR data) as related-but-distinct concepts.

3-5. Users, Roles, Permissions

HR staff/Owner/Admin (full CRUD) • Managers (scoped) • Employees (read/limited-edit of own record) • Super Admin (elevated read, audit-logged).

6. Features

- Employee Profile (personal, employment, structure assignment).
- Employee ↔ Membership linkage — optional in both directions.
- Employment Status lifecycle: Active, On Leave, Suspended, Terminated — distinct from Membership Lifecycle status and Organization Status.
- Document attachments via File Storage.
- Custom fields, org-configurable.
- Employee directory/search, scoped to viewer permission.
- Employee self-service view.

7. Workflows

Create: enter details → optionally link to a Membership → assign Structure Node → Employment Status = Active. Link to Membership later if invited — records merge by reference, never duplicated. Offboard: Employment Status → Terminated; record retained.

8. Edge Cases

Membership removed ≠ automatic termination. Assignment to an archived Structure Node blocked. Duplicate employee within one org prevented via org-scoped email uniqueness. Employee linked to a Membership from a different org impossible by construction.

9. Business Rules

Employee record always belongs to exactly one organization. Never hard-deleted — status-transitioned only. Employment Status kept fully independent from system access, Membership status, attendance status, leave status, payroll status.

10. Validation Rules

Required at creation: name, employment type, start date, status (defaults Active). Structure Node assignment must reference an active node in the same org. Org-scoped (not platform-wide) email uniqueness. Custom fields validated per configured type.

11. Security Considerations

Standard RLS. Field-level permission granularity built into the architecture from the beginning (only compensation fields, added by Payroll, use it in v1). All create/update/status-change actions audit-logged.

12-13. Multi-Tenant & Subscription

Employee records, custom field definitions, and structure assignments strictly organization-scoped. Gated by Billing; read-only-during-Locked.

14-15. Notifications/Reports

Record created/updated — audit-log-only. Basic directory/list view is core functionality; headcount analytics deferred to Analytics.

16. Future Scalability

Bulk import (CSV) • self-service portal expansion • compensation data (deferred to Payroll) • org chart visualization.

17. Dependencies

Organization Management (Structure Node) • RBAC • File Storage. Feeds Attendance, Leave, Payroll, Help Desk, Inventory, Finance.

18. Success Criteria

Create/view/edit/offboard without data loss • no cross-tenant leakage • downstream modules reference this record without duplicating it.

Additional Locked Architectural Decisions

Immutable internal Employee ID, independent of email or Membership. Future employment history support without redesign. Extensible profile. Business modules always reference the Employee ID, never duplicate employee information.

11.9 Module 7 — Attendance LOCKED

1. Purpose

Track when employees are present, absent, or remote — the time-and-presence data foundation Leave Management and Payroll both consume.

2. Business Problems Solved

Replaces manual/spreadsheet tracking with a structured, auditable record tied to the real Employee entity. Reliable worked-time input for Payroll. Real-time Manager visibility.

3-5. Users, Roles, Permissions

Employees (own entries) • Managers (approve, scoped) • Owner/Admin (full + policy config) • Super Admin.

6. Features

- Attendance Policy Configuration (working hours, work-week pattern, entry methods) per organization.
- Entry methods (configurable): Clock In/Out, Manual Entry, or hybrid.
- Attendance Status: Present, Absent, Half-Day, Remote/WFH, On Leave (read from Leave Management's write).
- Correction requests with Manager/Admin approval.
- Summary/history views, filterable.
- Regularization window — platform default 7 days, org-configurable.

7. Workflows

Daily entry → Manager review → correction request → approval → record updated (original + corrected retained). Leave Management writes "On Leave" status into Attendance via defined interface. Policy applies prospectively only.

8. Edge Cases

Clock-in without clock-out flagged "incomplete," never auto-closed. Correction outside window requires explicit Owner/Admin approval. No entry + no leave → "Absent" for follow-up. Approved Leave always takes precedence.

9. Business Rules

Entries never hard-deleted/overwritten — corrections preserve original alongside corrected value. Approved Leave always overrides Attendance status. Policy org-configurable, not hardcoded.

10-11. Validation & Security

Clock-out after clock-in. Manual entries need valid date/plausible duration. Correction requires non-blank reason. Employees see only own entries; corrections/approvals/policy changes audit-logged with before/after values.

12-13. Multi-Tenant & Subscription

Records/policies/corrections strictly organization-scoped. Gated by Billing; read-only-during-Locked.

14-15. Notifications/Reports

Correction requested/approved/rejected • incomplete-entry reminder. History view is core functionality; deeper analytics deferred.

16. Future Scalability

Geofencing/location-based clock-in • shift scheduling • biometric/device integrations • overtime rules (Payroll's domain).

17. Dependencies

Employee Management (Employee ID) • RBAC • Leave Management (writes "On Leave" status) • Payroll (consumes worked-time via stable interface).

18. Success Criteria

Entries recorded via enabled method(s) with no data loss • no silent alteration • approved leave always correctly overrides status.

Additional Locked Architectural Decisions

Immutable Attendance Record ID. UTC-internal calculations. Ready for future multiple work schedules. Policies version-aware. Stable interface for Payroll. Future check-in methods via provider adapters.

11.10 Module 8 — Leave Management LOCKED

1. Purpose

Own the full leave lifecycle as the authoritative source of leave decisions that Attendance reflects and Payroll consumes for pay-impact calculation.

2. Business Problems Solved

Replaces manual leave tracking with a structured, auditable workflow. Configurable leave policies (types, entitlements, approval chains). Automatic "On Leave" status in Attendance.

3-5. Users, Roles, Permissions

Employees (submit/track) • Managers (approve, scoped) • Owner/Admin (policy config, override) • Super Admin.

6. Features

- Leave Type Configuration per organization (regional variation).
- Entitlement/Accrual Policy per leave type, org-configurable math.
- Leave Balance Tracking (entitled/used/remaining).
- Leave Request Workflow with configurable approval chain.
- Leave Calendar View.
- Cancellation: pending cancels freely; approved cancellation requires same approval-chain review.
- Attendance Integration — publishes approval events.

7. Workflows

Submit → balance check → Pending → routed to approver → Approve (balance deducted, event to Attendance) / Reject. Cancel pending freely; cancel approved requires review, restores balance, retracts Attendance event. Accrual runs per configured policy. Policy changes prospective only.

8. Edge Cases

Overlapping date ranges rejected. Request exceeding balance rejected unless leave type explicitly permits negative. Approver removed mid-pending → escalates or Owner/Admin fallback. Deactivated leave type → historical records remain intact.

9. Business Rules

Approved leave always takes precedence in Attendance. Balances derived from immutable transactions, never directly editable. Policy changes prospective only. Cancelling approved leave requires same rigor as original request. No negative balances by default, configurable per-type.

10-11. Validation & Security

Date range valid, no overlap. Leave type active at submission. Balance sufficiency check. Whole-day only in v1. Employees see only own requests; all actions audit-logged; manual adjustments require mandatory reason.

12-13. Multi-Tenant & Subscription

Leave types/policies/requests/balances strictly organization-scoped. Gated by Billing; read-only-during-Locked.

14-15. Notifications/Reports

Submitted → approver • approved/rejected → employee • cancellation → relevant parties. Balance/history view and calendar are core functionality.

16. Future Scalability

Multi-step role-based approval chains • region-specific statutory compliance packs • half-day/hourly granularity • leave encashment.

17. Dependencies

Employee Management (Employee ID) • Attendance (publishes events into) • RBAC • Payroll (consumes finalized leave data only).

18. Success Criteria

Submit/track/cancel with accurate balance at all times • approved leave always correctly appears in Attendance • policy changes never retroactively corrupt history.

Additional Locked Architectural Decisions

Immutable Leave Request ID and Leave Type ID. Policies version-aware. Future regional compliance packs extend without modifying core rules. Leave history permanently available even after type deactivation.

11.11 Module 9 — Payroll LOCKED

Completes the launchable suite (Modules 1a-9). Calculate-and-record only in v1 — no disbursement/bank integration.

1. Purpose

Calculate and record employee compensation — consuming Employee, Attendance, and Leave data as inputs — to produce accurate, auditable payroll runs.

2. Business Problems Solved

Replaces manual/spreadsheet payroll calculation. Consolidates three data sources into one coherent calculation, without any upstream module knowing about payroll logic. Foundation for statutory compliance as swappable regional packs.

3-5. Users, Roles, Permissions

Owner/Payroll Admin (full access) • Manager (no default access, must be RBAC-granted) • Employee (view/download own payslips) • Super Admin.

6. Features

- Salary Structure Configuration (base pay, allowances, recurring deductions) per employee or template.
- Payroll Run for a defined pay period, using Attendance + Leave finalized data.
- Configurable deduction framework — not a built-in tax engine; Compliance Packs come later.
- Payslip Generation stored via File Storage.
- Approval Workflow: Draft → Reviewed → Approved → Finalized; a Finalized run is immutable.
- Payment Status Tracking only — no bank integration in v1.

7. Workflows

Configure salary structure → Run payroll (pull Attendance + Leave data → calculate gross/net → Draft) → Review → Approve → Finalize. Payslips generated on Finalization. Post-finalization corrections are always a new adjustment transaction in a subsequent run.

8. Edge Cases

Incomplete Attendance data → run flags employee for review. Pending leave excluded, flagged. Employee terminated mid-period → prorate to end date. Salary structure changed mid-period → run reflects version applied. Statutory rule changes → historical runs reflect rules in effect at the time.

9. Business Rules

A Finalized run is immutable — corrections only via new adjustment entries. Payroll never recalculates or overrides Attendance's or Leave's own data. Compensation data uses field-level access control. Payroll never source of truth for Employment Status/Attendance/Leave balances.

10-11. Validation & Security

A run cannot Finalize while any employee has an unresolved data-completeness flag. Salary structure fields non-negative, currency-aware. Compensation data requires strictest field-level access control. All changes audit-logged. Payslips inherit File Storage's signed-URL model.

12-13. Multi-Tenant & Subscription

Salary structures/runs/payslips strictly organization-scoped. Stricter Locked-state behavior: existing history viewable; running/approving/finalizing — including continuing a Draft — fully blocked while Locked.

14-15. Notifications/Reports

Run ready for review → Admin • finalized → employees • data-completeness flag → Admin. Payslip/payment history is core functionality; deeper analytics deferred.

16. Future Scalability

Full regional statutory Compliance Packs • bonus/incentive/commission structures • multi-currency payroll • direct disbursement provider integration.

17. Dependencies

Employee Management (field-level permissions) • Attendance (worked-time) • Leave Management (finalized leave) • RBAC • File Storage. **Consumed by Finance as a read-only finalized cost input.**

18. Success Criteria

Runs correctly consolidate Attendance/Leave data without overriding either • Finalized runs immutable, all corrections traceable • compensation protected by field-level permissions.

Additional Locked Architectural Decisions

Immutable Payroll Run ID and Payslip ID. Calculations transaction-based and fully reproducible. Every run captures immutable snapshots of Attendance/Leave/Employee/Salary Structure data used. Payroll components are configurable entities. Stable interfaces exposed for future Finance rather than direct integration. Future Compliance Packs plug in through extension points.

11.12 Module 10 — CRM LOCKED

First post-MVP module. Establishes Contact and Account as shared, platform-level Core Directory entities.

1. Purpose

Manage prospective and existing customer relationships — leads, contacts, accounts, deals/opportunities, interaction history.

2. Business Problems Solved

Structured lead-to-close tracking, replacing spreadsheets/disconnected tools. Establishes reusable Contact/Account concepts future modules (Help Desk, Finance, Projects, Marketplace) reuse instead of forking. Foundation for future sales reporting/forecasting.

3-5. Users, Roles, Permissions

Sales reps • Sales Managers • Owner/Admin (pipeline configuration) • Super Admin. Owner/Admin (full CRUD + config) • Manager (team-scope) • Employee (own assigned, RBAC-configurable broader).

6. Features

- Contact & Account Management — Contact may link to an Account or stand alone.
- Lead Management with Lead Status (New, Contacted, Qualified, Disqualified, Converted).
- Deal/Opportunity Pipeline with org-configurable stages.
- Activity/Interaction Log (immutable, append-only history).
- Task/follow-up reminders tied to a Contact/Deal.
- Lead Conversion — one action creates Contact + Account + Deal, preserving history.
- Document attachments via File Storage.

7. Workflows

Capture lead → qualify/convert (once, never re-converted) → progress through configurable pipeline stages, each change logged → close (Won/Lost). Activities logged at any point. Reassignment logged.

8. Edge Cases

Double-conversion prevented by status transition. Backward pipeline moves permitted and logged. Contact linked to later-deleted Account — block-or-archive pattern. Employee removed while holding leads/deals — reassignment required or flagged.

9. Business Rules

Pipeline stages organization-configurable, never hardcoded. Records never hard-deleted. A Converted Lead cannot be re-converted. Activity log entries immutable once created.

10-11. Validation & Security

A Deal must link to at least one Contact or Account. Required: name, at least one contact method. Standard RLS; visibility scoping via RBAC; all creates/updates/stage-changes audit-logged.

12-13. Multi-Tenant & Subscription

Contacts/Accounts/Leads/Deals/Activities strictly organization-scoped. Gated by Billing; standard read-only-during-Locked.

14-15. Notifications/Reports

Lead/deal reassigned → new owner • deal Closed Won/Lost → Manager/Admin. Pipeline/deal list core functionality; forecasting deferred.

16. Future Scalability

Email integration • lead-source/marketing attribution • sales forecasting/quotas • hand-off to Finance on Closed Won • bulk import.

17. Dependencies

RBAC • File Storage • Employee Management (ownership ref). **Consumed by Help Desk (Contact/Account reuse), Projects (optional Account link), Finance (Closed Won invoicing input), Marketplace (future).**

18. Success Criteria

Full lead-to-close tracking with accurate history • no record hard-deleted or orphaned • pipeline fully org-configurable • ownership always traces to a valid Employee ID.

Additional Locked Architectural Decisions

Contact and Account are shared, platform-level business entities (same principle as Employee and Structure Node) — CRM owns their lifecycle; future Help Desk, Finance, Projects, Marketplace reuse them through stable interfaces. Immutable internal IDs on every Lead, Contact, Account, Deal, Activity. Activities are immutable interaction records. Deal probability, forecasting, commissions, revenue recognition, quotations, and invoicing remain outside CRM v1.

11.13 Module 11 — Projects LOCKED

1. Purpose

Manage internal and client-facing work — projects, tasks, milestones, and time tracking — giving future Finance a basis for project-based billing.

2. Business Problems Solved

Replaces scattered task lists/spreadsheets with a structured project → task hierarchy tied to real Employees and, for client work, CRM's shared Account entity.

3-5. Users, Roles, Permissions

Project Managers/Team Leads • Team Members/Employees • Owner/Admin • Super Admin. Project Member concept — project-scoped, distinct from RBAC.

6. Features

- Project: name, description, optional linked Account, dates, status (Planning, Active, On Hold, Completed, Cancelled).
- Task: org-configurable status set, assignable to one or more Employees, due date, priority, optional parent (subtasks).
- Milestone: dated reference marker, no independent workflow.
- Time Tracking (optional per org) — raw time only, never billing/payroll calculations.
- Project Member management.
- Comments/Activity Feed per task.
- Document attachments via File Storage.

7. Workflows

Create project → add Project Members → Planning/Active. Create/assign task → status updates logged. Time logged if enabled. Complete/close project never auto-completes remaining tasks.

8. Edge Cases

Employee removed while holding open tasks → reassignment required or flagged. Task Done while parent/project isn't → allowed. Subtask under a deleted task → block-or-reassign. Project linked to archived CRM Account unaffected.

9. Business Rules

Projects/Tasks never hard-deleted. Task status sets org-configurable. Project Membership grants visibility/assignment eligibility within that project only, not a formal RBAC construct.

10-11. Validation & Security

A Task must belong to exactly one Project. Assignees must be current Project Members unless org allows any Employee. Standard RLS; project-scoped visibility layered under org RBAC; all actions audit-logged.

12-13. Multi-Tenant & Subscription

Projects/Tasks/Milestones/Time strictly organization-scoped. Gated by Billing; standard read-only-during-Locked.

14-15. Notifications/Reports

Task assigned/reassigned • due date approaching/overdue • project status changed • comment added. Project/task list core functionality; deeper analytics deferred.

16. Future Scalability

Gantt/timeline and dependency tracking • Kanban visualization • budget tracking (Finance tie-in) • recurring tasks/templates • bulk import.

17. Dependencies

RBAC • CRM (optional Account reference) • File Storage • Employee Management (task assignment). **Consumed by Finance (time-tracking as future billing input).**

18. Success Criteria

Teams plan/assign/track work to completion with full accurate history • task status sets fully org-configurable • client-linked projects correctly reference CRM's shared Account.

Additional Locked Architectural Decisions

Project Task and Help Desk Ticket remain deliberately separate business entities — different domains, lifecycles, workflows; future integration via stable interfaces only, not a shared underlying entity. Immutable internal IDs on every Project, Task, Milestone, Time Entry, Comment, Project Membership. Status configurations version-aware. Project archiving preserves all historical data.

11.14 Module 12 — Help Desk LOCKED

1. Purpose

Manage customer and/or internal support requests — tickets, their lifecycle, assignment, and resolution — reusing CRM's shared Contact/Account entities for "who is asking" rather than duplicating customer data.

2. Business Problems Solved

Structured support workflow (replacing shared inboxes/spreadsheets) for external customer support and, optionally, internal IT/HR-style support. Reuses the platform's existing Contact/Account model instead of a second, disconnected customer record. Ticket-based SLA/response tracking as a foundation for future support analytics.

3. Users Involved

Support Agents/Employees with Help Desk access • Support Managers/Team Leads • Requesters: external Contacts (via CRM's shared entity) or internal Employees • Organization Owner/Admin (category/SLA configuration) • Platform Super Admin.

4. User Roles

No new identity roles. RBAC-governed on existing coarse roles. A lightweight Team/Queue concept (which agents handle which category) is help-desk-scoped, similar in spirit to Projects' Project Membership — distinct from RBAC.

5. Permissions

Owner/Admin: full access, category/SLA configuration • Support Manager: full visibility/management within their team's queue(s), reassignment, escalation • Support Agent: manage tickets assigned to them; broader queue visibility RBAC-configurable • Requester: view/comment on their own submitted tickets (light in v1) • Platform Super Admin: elevated read, audit-logged.

6. Features

- Ticket: subject, description, org-configurable status set (New, Open, Pending, Resolved, Closed), priority, category, linked Requester (CRM Contact/Account or internal Employee).
- Assignment to an Agent or a Team/Queue.
- Comments/Responses — internal notes (agent-only) vs. external-facing responses (visible to requester), a hard distinction, **structurally separate entities, not merely differentiated by a visibility flag.**
- Category/Type Configuration, org-defined.
- Basic SLA Tracking — response target + resolution target + overdue indicator only, per priority/category.
- Manual escalation (reassignment to a Manager/higher tier).
- Document attachments via File Storage.

7. Workflows

Create ticket → queue at "New," auto-assign per category rules or manual pickup → Work ticket → Open → internal notes/external responses as work progresses → Resolve → Closed. Reopen → logged, not a fresh disconnected ticket. Reassign/escalate → logged. SLA breach → flagged overdue, a visibility signal only in v1.

8. Edge Cases

Requester Contact later archived — ticket remains intact and readable. Agent removed from org while holding open tickets — reassignment required or flagged, never orphaned. No category selected — falls into a default/general category, not blocked. Internal note accidentally sent externally — flagged as a hard structural boundary, not implementation-level discretion. Ticket reopened long after closure — permitted, visibly flagged as "reopened."

9. Business Rules

Tickets and comment/response history never hard-deleted — status-based lifecycle. Internal notes are never visible to the requester under any circumstance — a hard boundary, not RBAC-configurable. Ticket status sets are organization-configurable, never hardcoded platform-wide. Reassignment and status-change history is immutable/append-only.

10. Validation Rules

A Ticket must have a subject/description and a linked Requester. Required at creation: subject, status (defaults New).

11. Security Considerations

Standard RLS tenant isolation. Internal-note visibility enforced structurally (separate entity), not just by UI convention. All creates/updates/status-changes/reassignments/comments audit-logged. Attachments inherit File Storage's signed-URL, tenant-isolated model.

12. Multi-Tenant Considerations

Tickets, Categories, and Comments strictly organization-scoped. Referenced Contacts/Accounts remain governed by CRM's own tenant-isolation rules.

13. Subscription Considerations

Gated by Billing's centralized check. Standard read-only-during-Locked — existing tickets/history viewable, new

tickets/comments/status-changes blocked.

14. Notifications

Ticket (re)assigned → new Agent • new external-facing response → requester • SLA breach/overdue → Agent/Manager • Resolved/closed → requester.

15. Reports

Ticket queue/list views are core functionality, not reports. Resolution-time trends, agent performance, SLA compliance deferred to future Analytics/Reports.

16. Future Scalability

Full self-service customer portal (explicitly deferred, see below) • rule-based auto-escalation • knowledge base/FAQ deflection • live chat/omnichannel intake • CSAT surveys post-resolution.

17. Dependencies

CRM: Requester references shared Contact/Account (referenced, never duplicated) • Employee Management: internal-ticket requesters and Agent assignment reference Employee ID • RBAC, File Storage, Notification Framework, Audit Logging, Billing.

Consumed by Analytics (reporting interface) and the Platform AI Layer (future AI-assisted responses).

18. Success Criteria

Requests captured, assigned, worked, resolved with full accurate history • internal notes never exposed to requesters, ever • no ticket data hard-deleted or silently orphaned • status/category sets fully org-configurable • no cross-tenant data; Contact/Account references stay correctly linked.

Locked Decisions:

Self-Service Portal: Deferred entirely to Future Scalability. v1 is internal-agent-managed only; external requester authentication, customer portals, and self-service experiences are separate future platform capabilities, built without requiring changes to the Help Desk core domain (external Contacts are not platform Users/Memberships — a genuinely new identity concept, not a v1 add-on).

SLA Management: v1 = Response target + Resolution target + Overdue indicator only. Automated routing, escalations, business calendars, holiday awareness, reminder workflows, and SLA automation are all future capabilities.

Additional Locked Architectural Decisions

Immutable internal IDs on every Ticket, Comment, Category, Queue, Assignment, and SLA Policy. Ticket history is event-based and immutable — every lifecycle transition permanently traceable. Attachments always routed through the centralized File Storage module. Internal notes and customer-visible responses are structurally separate entities. Future omnichannel integrations (Email, WhatsApp, Live Chat, Voice, Social Media) connect through provider abstractions without touching Help Desk business logic. Future Knowledge Base is an independent module that Help Desk references, never embeds. Ticket templates, canned responses, automation rules, CSAT surveys, AI-assisted responses, and chatbot capabilities are all future extensibility items. Help Desk exposes stable interfaces for future Analytics and the Platform AI Layer without exposing internal implementation details.

11.15 Module 13 — Inventory LOCKED

1. Purpose

Track stock — items, quantities, movements, and valuation — as the platform's authoritative source for "what do we have and where," giving future Finance a clean cost/valuation input and future Projects/CRM a reference point for product-linked work, without any of those modules forking their own stock data.

2. Business Problems Solved

Replaces spreadsheets/manual counts with a structured, auditable stock ledger. Prevents overselling/understocking via real-time on-hand visibility. Establishes Item/SKU as a shared, reusable entity for future Finance (COGS, invoicing) and Marketplace (future product catalog), without embedding financial logic here.

3. Users Involved

Inventory/Warehouse Staff • Inventory Managers • Owner/Admin (item/category/location configuration) • Employees in other modules who reference stock (read-only consumers) • Platform Super Admin.

4. User Roles

No new identity roles. RBAC-governed on existing coarse roles.

5. Permissions

Owner/Admin: full access, item/category/location configuration • Inventory Manager: full stock operations across assigned location(s) • Inventory Staff: record movements within assigned location(s); no configuration access • Other-module Employees: read-only stock lookups where RBAC-granted • Platform Super Admin: elevated read, audit-logged.

6. Features

- Item (SKU): name, SKU code (org-unique, editable), category, unit of measure, optional description, active/inactive status.
- Location: one or more stock-holding locations per org — flat list in v1, not a hierarchy.
- Stock Level: current on-hand quantity per Item per Location, always derived from the movement ledger — never a directly editable field.
- Stock Movements (the immutable ledger): Receipt (stock in), Issue (stock out), Transfer (location to location), Adjustment (correction, requires reason).
- Reorder Point per Item (optional), flags low-stock, visibility signal only — no automated purchasing in v1.
- Low-Stock Alerts via Notification Framework.
- Document attachments via File Storage.

7. Workflows

Item created → assigned category/UOM → Active. Receipt recorded → stock level increases. Issue recorded → stock level decreases (blocked if it would go negative, unless explicitly configured to allow it per-Item). Transfer recorded → simultaneous decrease at source / increase at destination, single atomic movement pair. Adjustment recorded → requires a mandatory reason, always audit-logged. Reorder Point breached → low-stock notification fired.

8. Edge Cases

Issue exceeding on-hand quantity blocked by default; per-Item override never silent. Item deactivated while stock remains — deactivation blocks new transactions but never hides existing stock/history. Transfer to same source/destination rejected. Location removed while holding stock blocked. Simultaneous movements resolved via the ledger's append-only, transaction-based design.

9. Business Rules

Stock level is never a directly editable field — always the sum of the immutable movement ledger. Movements never edited or deleted — corrections are new, offsetting Adjustment entries. Items and Locations never hard-deleted. Reorder Point is a visibility signal only.

10. Validation Rules

Item requires: name, SKU (unique within org), unit of measure. Every movement requires: Item, Location(s), quantity (positive, non-zero), movement type. Adjustment requires non-blank reason. Transfer requires two distinct Locations.

11. Security Considerations

Standard RLS tenant isolation. All movements audit-logged with actor, timestamp, before/after quantity. Attachments inherit File Storage's signed-URL model.

12-13. Multi-Tenant & Subscription

Items, Locations, Movements, Reorder Points strictly organization-scoped. Gated by Billing; standard read-only-during-Locked.

14-15. Notifications/Reports

Low-stock → Inventory Manager/Admin • stock adjustment → audit-log-only • transfer completed → optional. Current stock level and movement history are core functionality; valuation trends/turnover/shrinkage analytics deferred.

16. Future Scalability

Serial/batch/lot number tracking • multiple valuation methods (deferred to Finance) • Purchase Order/supplier management • barcode/QR scanning • multi-level location hierarchy.

17. Dependencies

Employee Management (movement actor reference) • File Storage, Notification Framework, Audit Logging, RBAC, Billing. **Consumed by Finance (valuation/COGS input), Projects (future material consumption), Marketplace (future catalog).**

18. Success Criteria

Stock levels always mathematically consistent with the movement ledger, no drift • no movement ever edited/deleted after creation • no item/location hard-deleted or silently orphaned while holding stock • low-stock visibility accurate and timely.

Locked Decisions:

Inventory Valuation: Fully deferred to Finance (Module 14) / future Inventory extensions. v1 = quantity and movement tracking only. No unit cost, FIFO/LIFO/Weighted Average, or COGS anywhere in Inventory v1.

Negative Stock: Blocked by default; explicit, disabled-by-default per-Item override for orgs that need it.

Additional Locked Architectural Decisions

Immutable internal IDs on every Item, SKU, Location, Stock Movement, Category, Reorder Policy. SKU codes unique-within-org but editable; Item ID is the permanent, immutable system reference. Stock quantities derived exclusively from the immutable movement ledger — no module may directly write a balance. Inventory policies version-aware. Future serial/batch/lot tracking extends the Item model without redesigning the movement ledger. Future Supplier/PO/SO/Manufacturing/Procurement modules reference Inventory through stable interfaces, never duplicate stock data. Physical counts/reconciliation implemented later as Adjustment transactions. Stable quantity/movement interfaces exposed to Finance, Projects, Marketplace, and future modules.

11.16 Module 14 — Finance LOCKED

1. Purpose

Own core financial record-keeping — invoicing, expenses, and the accounting layer that consumes finalized data from Payroll, CRM (Closed Won deals), Projects (time-billing), and Inventory (movements) — as the platform's single source of financial truth, without embedding accounting logic into any upstream module.

2. Business Problems Solved

Replaces disconnected invoicing/expense spreadsheets with a structured, auditable financial ledger. Consolidates inputs from Payroll (compensation cost), CRM (deal-to-invoice), Projects (billable time), and Inventory (stock movement/valuation) into one coherent financial picture — without any of those modules knowing accounting rules. First module to introduce real monetary valuation (inventory costing) and basic bookkeeping to the platform.

3. Users Involved

Finance/Accounting staff • Owner/Finance Admin (via RBAC grant) • Sales reps (viewing invoice status, read-only) • Employees submitting expenses • Managers approving expenses • Platform Super Admin.

4. User Roles

No new identity roles. Second module (after Payroll) where Employee Management's field-level permission capability is actively used, given financial data sensitivity.

5. Permissions

Owner/Finance Admin: full access — invoicing, expenses, chart of accounts configuration, inventory valuation settings • Manager: approve expense claims within scope; no invoicing/COA access by default • Employee: submit own expenses, view own claim status • Sales rep: read-only view of invoice status linked to their own deals • Super Admin: elevated read, audit-logged.

6. Features

- Invoice: linked to CRM Account/Contact (and optionally a Closed Won Deal or Project), line items, tax fields (configurable reference data, not a tax engine), status (Draft, Sent, Paid, Overdue, Void).
- Expense Claim: submitted by Employee, category, amount, attachment (receipt via File Storage), Manager approval workflow.
- Chart of Accounts — simple, org-configurable list of account categories (categorization only, not full double-entry ledger).
- Inventory Valuation — unit cost capture on Inventory Receipts (via Inventory's stable interface), Weighted Average as the sole v1 valuation method, COGS calculation on Issue movements.
- Payment Recording — marks invoices/expenses as paid; no payment gateway integration in v1.
- Document attachments via File Storage.

7. Workflows

Invoice created (manually, or from a Closed Won Deal/completed Project) → Draft → Sent → Paid/Overdue tracked. Expense submitted → Manager approval → Approved/Rejected → Payment recorded separately. Inventory Receipt with unit cost → valuation ledger updated → Issue movements draw COGS per Weighted Average. Period-end view aggregates Invoices + Expenses + Payroll cost + Inventory COGS into a basic financial summary (not full GL/reporting — deferred to Analytics).

8. Edge Cases

Invoice linked to a CRM Account that's later archived — invoice remains intact, reference preserved. Expense submitted with no receipt — allowed but org-configurable whether mandatory. Inventory Issue occurring before any costed Receipt exists — flagged for review, never silently defaulted to zero cost. Invoice voided after Paid — requires explicit reason, creates offsetting record, never deletes original. Payroll run finalized mid-period — Finance consumes it as a locked snapshot, never recalculates.

9. Business Rules

Financial records never hard-deleted — status/void-based. Finance never overrides or recalculates Payroll, CRM, Projects, or Inventory data — strictly a downstream, read-only consumer. Inventory valuation entries follow the same immutable-ledger pattern — corrections are new offsetting entries, never edits. Chart of Accounts changes apply prospectively only.

10. Validation Rules

Invoice requires: linked Contact/Account, at least one line item, currency. Expense requires: amount, category, submitting Employee. Unit cost on Inventory Receipt required if Inventory Valuation is enabled.

11. Security Considerations

Standard RLS tenant isolation. Financial data uses the platform's strictest field-level access control, same tier as Payroll compensation data. All create/update/status-change/void actions audit-logged. Attachments inherit File Storage's signed-URL model.

12-13. Multi-Tenant & Subscription

Invoices, Expenses, Chart of Accounts, Valuation entries strictly organization-scoped. Gated by Billing; stricter Locked-state behavior consistent with Payroll — existing history/invoices/expenses viewable; creating, approving, sending, or voiding anything financial fully blocked while Locked.

14-15. Notifications/Reports

Invoice sent/overdue → Finance Admin and, optionally, linked sales rep • expense submitted/approved/rejected → relevant parties • invoice Paid → Finance Admin. Invoice/expense list and basic period summary are core functionality; deeper financial analytics deferred to Analytics.

16. Future Scalability

Full double-entry general ledger • payment gateway integration • multi-currency financial consolidation • FIFO/LIFO inventory valuation methods • recurring/subscription invoicing • tax engine/regional compliance packs.

17. Dependencies

CRM (Closed Won → Invoice input) • Projects (billable time, future) • Payroll (finalized run cost) • Inventory (movement ledger for valuation/COGS) • Employee Management (expense submitter, field-level pattern) • RBAC, File Storage, Notification Framework, Audit Logging, Billing.

18. Success Criteria

Invoices and expenses tracked with full accurate history, nothing hard-deleted • Finance never mutates upstream module data — strictly consumes finalized snapshots • Inventory valuation mathematically consistent with the movement ledger.

Locked Decisions:

Payment Gateway: None in v1. Finance = record-keeping only (Invoice creation, Payment recording, Payment status tracking, Expense recording, Financial history). Same "calculate-and-record only" boundary as Payroll.

Inventory Valuation: Weighted Average only in v1. FIFO, LIFO, Standard Cost, Moving Average deferred; valuation engine architected so future methods plug in without redesigning Finance or Inventory.

Expense Receipts: Org-configurable — Mandatory or Optional-but-recommended, not hardcoded platform-wide.

Additional Locked Architectural Decisions

Immutable internal IDs on every Invoice, Expense Claim, Payment Record, Valuation Entry, Chart of Account, Financial Period. Financial records follow an immutable transaction model — corrections always reversing/adjustment transactions. Finance consumes finalized snapshots from Payroll, Inventory, CRM, Projects through stable interfaces only. Financial Periods support future period locking. Currency handling provider-independent, multi-currency-ready. Tax rates configurable reference data only — no tax engine in v1. Future GL, AR, AP, Budgeting, Fixed Assets, Bank Reconciliation, Financial Compliance modules extend Finance via stable interfaces. Finance exposes stable interfaces for Analytics and the Platform AI Layer.

11.17 Module 15 — Analytics & Reports LOCKED

1. Purpose

Provide cross-module reporting and dashboards — read-only aggregation and visualization over data owned by every other module — without becoming a second source of truth for any business fact.

2. Business Problems Solved

Fulfills every prior module's explicitly deferred analytics item (headcount, resolution-time, forecasting, cost trends, P&L, turnover, etc.) in one coherent place. Gives Owner/Admin cross-functional visibility no single module can provide alone. Foundation for future compliance/audit-style reporting and the AI Platform Layer's need for structured, queryable business context.

3. Users Involved

Owner/Admin (full cross-module visibility) • Managers (scoped dashboards, per existing module permissions) • Employees (personal/scoped views) • Platform Super Admin.

4. User Roles

No new identity roles. RBAC-governed — a user's Analytics visibility into any dataset can never exceed what RBAC already grants them in the owning module.

5. Permissions

Owner/Admin: full cross-module dashboard access, report builder access • Manager: scoped dashboards matching existing per-module scope • Employee: personal-scope views only • Platform Super Admin: elevated read, audit-logged.

6. Features

- Dashboard — configurable widgets (charts, KPIs, tables) pulling from one or more modules' exposed reporting interfaces.
- Standard Report Library — pre-built reports realizing every module's deferred item.
- Cross-Module Reports — a small set intentionally joining data across modules (e.g., Project cost = Payroll cost + Finance expense).
- Ad-hoc Report Builder — filter/group/aggregate/sort over exposed module data. No raw SQL/direct database access.
- Export — CSV only in v1.
- Scheduled Report Delivery via Notification Framework.

7. Workflows

Module finalizes/updates data → exposes it through its own stable reporting interface → Analytics reads it on demand or refresh cycle. User opens a Dashboard → widgets resolve per-user RBAC scope. Report generated → optionally scheduled → delivered via Notification Framework. Ad-hoc report saved, org-scoped, bound by requester's RBAC scope at run time, not creation time.

8. Edge Cases

User's RBAC access revoked after a dashboard/report was built — re-evaluated at every run, never cached with stale permissions. Module with no data yet — empty state, never errors. Cross-module report where one side is Locked — that contribution shows as unavailable/stale-flagged, doesn't block the rest. Very large date ranges flagged for future async generation. Archived source record still appears correctly in historical reports.

9. Business Rules

Analytics never becomes a second source of truth — always reads through each module's own exposed reporting interface. A user can never see more than their RBAC access already permits. Reports reflect point-in-time data as reported by the owning module. Standard reports read-only; Ad-hoc builder view/filter/aggregate only, never a write path.

10. Validation Rules

Ad-hoc reports require at least one data source and one metric/dimension. Scheduled delivery requires a valid recipient and frequency.

11. Security Considerations

Standard RLS tenant isolation. RBAC re-evaluated at report run time, not cached from creation time. Report generation/export/schedule actions audit-logged.

12-13. Multi-Tenant & Subscription

Dashboards/reports/schedules strictly organization-scoped, including Super Admin platform-wide views (separate, elevated, audit-logged). Gated by Billing; read-only-during-Locked applies naturally.

14-15. Notifications/Reports

Scheduled report ready/delivered → recipient • generation failure → requester. This module is the reports layer — no further deferral point above it.

16. Future Scalability

Async/background generation • predictive analytics/forecasting (tie-in to Platform AI Layer) • custom visualization/drill-down • data warehouse migration path • scheduled export to external BI tools.

17. Dependencies

Every business module (each exposes its own stable reporting interface) • RBAC (visibility at run time) • Notification Framework (scheduled delivery) • Billing (module gate). **Future Platform AI Layer: Analytics' structured interfaces are a natural future data source for AI-assisted insights.**

18. Success Criteria

Every module's deferred analytics item realized here, no module needs parallel reporting logic • no report ever exposes data beyond viewer's RBAC scope • Analytics never becomes a write path or duplicate data store.

Locked Decisions:

Ad-hoc Report Builder: No raw SQL or direct database access for Version 1. Strictly filtering, grouping, aggregation, sorting, visualization over each module's exposed reporting interfaces.

Export Formats: CSV as the only export format for Version 1. PDF, Excel-specific formatting, scheduled exports to external BI tools, and advanced reporting integrations remain future capabilities.

Additional Locked Architectural Decisions

Immutable internal IDs on every Dashboard, Report Definition, Widget, Scheduled Report, Report Template. Analytics remains a read-only platform capability, never a secondary source of business data. All report calculations reproducible from source modules without independent business logic. Dashboard definitions metadata-driven. Long-running report generation architected for future async execution. Report scheduling references Notification Framework through stable interfaces. Future external BI integrations consume Analytics through stable APIs. Analytics exposes stable interfaces to the Platform AI Layer for future AI insights, summaries, forecasting, and natural language querying without coupling Analytics to any AI provider.

11.18 Module 16 — Platform AI Layer LOCKED

Architectural framing: designed as a shared Platform AI Layer, not an isolated business module. Business modules consume AI capabilities through platform interfaces rather than embedding AI logic internally — matching the pattern every other platform-service module (Notifications, Audit Logging, File Storage) already follows.

1. Purpose

Provide a single, provider-agnostic AI capability layer — text generation, summarization, classification, extraction, and (future) conversational/agentic actions — that every business module consumes through stable platform interfaces. No business module embeds its own AI provider integration or prompt logic.

2. Business Problems Solved

Prevents every future "AI-assisted X" feature (Help Desk's AI-assisted responses, CRM's future timeline summarization, Analytics' NL querying) from independently integrating a provider and duplicating prompt/cost/safety logic. Centralizes AI provider vendor risk behind one abstraction, consistent with Vendor Independence. Single place to enforce tenant isolation, RBAC, audit logging, and subscription gating for AI usage.

3. Users Involved

Indirect only in v1 — end users interact with AI through a calling business module's UI, never a standalone "AI module" screen • Owner/Admin (AI feature enablement per org, provider/cost visibility) • Platform Super Admin (provider configuration, usage monitoring) • Every future/present business module (the actual API consumers).

4. User Roles

No new identity roles. RBAC-governed via a capability-check pattern (e.g., `ai.use`, scoped per feature) — a calling module's AI-powered feature still requires the user to hold whatever permission that module already defines for the underlying action. The AI Layer never grants access a business module wouldn't already grant.

5. Permissions

Owner/Admin: enable/disable AI features per org, per module; view AI usage/cost • Business-module users: AI-assisted actions available only where they already hold the underlying business-module permission • Platform Super Admin: provider configuration, cross-org usage monitoring, audit-logged.

6. Features

- AI Provider Abstraction — a single internal interface (e.g., `generate(prompt_context, capability_type)`) that every module calls; concrete provider swappable behind it with zero calling-module changes.
- Capability Types (v1, deliberately narrow): Text Summarization, Text Generation/Drafting, Classification/Tagging — no agentic/autonomous action-taking in v1.
- Per-Module, Per-Org Feature Flags — Owner/Admin enables specific AI capabilities per business module independently.
- Usage Tracking — request counts/tokens per org, per module, per capability.
- Prompt/Context Assembly Contract — each calling module supplies structured context; the AI Layer never independently queries other modules' data.
- Content Safety Guardrails — basic input/output filtering hook, provider-agnostic, extensible.

7. Workflows

Business module triggers an AI action → assembles permitted context per its own RBAC-scoped data → calls the AI Layer's stable interface with a capability type → AI Layer resolves active provider → returns result → calling module handles display/storage in its own domain. Owner/Admin enables/disables an AI capability for their org → immediately reflected, no code changes required. Provider outage/failure → clear failure signal → calling module degrades gracefully, never a blocked core action.

8. Edge Cases

AI feature disabled for an org — calling module's AI-assisted UI element simply doesn't appear/execute. Provider timeout/error never blocks or corrupts the underlying business action. Module requests an ungranted capability type — rejected at the interface level, fails closed. Sensitive data (Payroll compensation, Finance figures) passed as AI context — calling module remains responsible for only supplying RBAC-permitted data. AI output containing unsafe content — surfaced through the guardrail hook, calling module decides how to handle.

9. Business Rules

The AI Layer never embeds business logic — no knowledge of Tickets, Deals, or Payslips, only generic capability types and structured context. The AI Layer never grants access beyond what the calling module's own RBAC check already permits. No module may call an AI provider SDK directly. AI-generated content is never treated as ground truth — always assistive/suggested, subject to the same human review workflows the module already has. Fail closed: unconfigured/disabled capability requests are denied, never silently degraded.

10. Validation Rules

Every AI request requires: capability type, organization_id, calling module identifier, structured context payload. Requests to a

disabled or unconfigured capability rejected before reaching any provider.

11. Security Considerations

Standard RLS tenant isolation applies to usage records and configuration. AI requests/responses are NOT persisted by the AI Layer itself beyond usage metering metadata — content persistence is the calling module's responsibility. All enable/disable and provider configuration changes audit-logged. Context payloads never logged in full within Audit Logging — only capability type, org, module, and outcome metadata.

12. Multi-Tenant Considerations

Feature flags, usage records, and AI-Layer-owned configuration strictly organization-scoped; provider credentials/configuration are platform-level (Super Admin), never per-tenant secrets managed by individual orgs.

13. Subscription Considerations

AI capabilities gated by Billing at two levels: (a) the calling business module must itself be active/subscribed, and (b) AI features may additionally require their own future entitlement (flagged, not solved here). Locked orgs cannot trigger new AI actions.

14. Notifications

Not a primary source in v1. Provider outage/degraded-service state may surface to Platform Super Admin via existing notification channels; not user-facing per org in v1.

15. Reports

AI usage volume is a foundation for future cost/analytics reporting, deferred to Analytics & Reports once this module has real usage data to expose.

16. Future Scalability

Agentic/autonomous multi-step AI actions (explicitly out of scope in v1) • conversational/chat interfaces • fine-tuned/custom models per org • natural-language querying surface for Analytics • cost-based entitlement/usage billing • multi-provider routing.

17. Dependencies

RBAC (calling modules enforce their own checks; AI Layer performs the capability-level check only) • Billing (module-subscription gate, future AI-specific entitlement) • Audit Logging (metadata-only events) • Notification Framework (Super Admin operational alerts). **Consumed by Help Desk (future AI-assisted responses), CRM (future timeline/summarization), Analytics (future NL querying/insights), and any future business module.**

18. Success Criteria

No business module ever integrates an AI provider SDK directly • switching AI providers requires zero calling-module changes • AI Layer never expands any user's access beyond their existing RBAC-scoped permissions • AI feature failures never block or corrupt core business actions • no cross-tenant data ever crosses through AI context assembly.

Locked Decisions:

AI Provider: Fully provider-agnostic. v1 default implementation philosophy is **self-hosted open-source models (via Ollama or vLLM)** as the preferred architecture, consistent with the platform's free-tier and vendor-independence principles. Cloud providers (Anthropic Claude, OpenAI, Google Gemini, Groq, OpenRouter, or other compatible providers) are implemented as interchangeable adapters when required. No business module ever knows which provider is active.

v1 AI Capabilities: Text Summarization, Text Generation/Drafting, Classification/Tagging only. Agentic workflows, autonomous actions, long-term memory, reasoning chains, tool orchestration, conversational assistants, and workflow automation remain future capabilities.

Content Persistence: None by the Platform AI Layer. Only operational metadata stored: Request ID, Organization ID, Calling Module, Capability Type, Provider, Token Usage, Processing Time, Success/Failure Status. Any AI-generated content worth retaining is stored only by the calling business module under its own rules.

Additional Locked Architectural Decisions

Immutable internal IDs on every AI Request, AI Capability, AI Configuration, AI Provider, AI Usage Record. AI providers implemented through adapters following the platform's existing provider abstraction pattern. Prompt templates version-aware — future improvements never change historical behavior. The AI Layer never directly accesses databases or business modules, operating only on structured context explicitly provided. AI requests support future asynchronous execution without changing public interfaces. Future RAG, Vector Search, MCP integrations, Tool Calling, Agent Frameworks, and Model Routing extend the AI Layer without redesigning the core. Stable interfaces exposed for every future business module while remaining completely independent of any specific AI provider.

11.19 Module 17 — Marketplace LOCKED

1. Purpose

Extend the platform's existing internal Module Registry & Activation Model (owned by Organization Management) into a customer- and third-party-facing extensibility surface — allowing first-party modules to be discovered/activated through a proper storefront experience, and laying the foundation for future third-party-built extensions, without redesigning how modules are built or activated today.

2. Business Problems Solved

Turns the already-existing internal module registry into something Owners/Admins can actually browse, compare, and activate from. Establishes a structured extension-point model so a future third-party developer ecosystem is possible without retrofitting every existing module. Creates a single place to surface module dependencies before activation.

3. Users Involved

Organization Owner/Admin (browses, activates, deactivates) • Platform Super Admin (curates the catalog, approves listings) • Future third-party developers (publish extensions — architecture only, not built in v1) • Platform team (publishes first-party modules/packs into the catalog).

4. User Roles

No new identity roles. Module activation itself remains an Owner/Admin action per Organization Management's existing locked rule.

5. Permissions

Owner: activate/deactivate any listed module or pack (same authority Org Management already granted) • Org Admin: browse catalog; activation rights per existing RBAC/Org Management rules • Platform Super Admin: curate/publish/unpublish catalog listings, approve future third-party submissions • Future third-party developer: submit listings for review (architecture placeholder, not implemented in v1).

6. Features

- Catalog Listing — one entry per activatable unit: a core Business Module, or a future sub-unit like a Payroll Compliance Pack or an AI capability bundle.
- Listing Metadata — name, description, category, dependency declarations (reusing the Module Dependency Matrix), pricing reference (reads from Billing, never owns pricing itself).
- Activation Flow — surfaces the existing Module Registry & Activation Model (Org Management/Billing-owned) through a browsable UI; Marketplace itself never performs activation logic.
- Dependency Surfacing — shows what a module needs/provides before activation, using the platform's locked Dependency Matrix as source data.
- Future Third-Party Extension Points (architecture only, v1 scope: definition, not implementation) — a conceptual "Extension" entity distinct from a first-party "Module," with its own review/approval state machine.

7. Workflows

Owner/Admin browses Marketplace catalog → views a listing's dependencies/pricing → triggers activation → Marketplace hands off to the existing Billing/Org Management activation interface (no new activation logic invented here) → module becomes active exactly as it does today. Platform team publishes a new first-party module/pack into the catalog as it's finalized. (Future) third-party developer submits an Extension → Pending Review state → Platform Super Admin approves/rejects.

8. Edge Cases

Listing activated whose dependency isn't yet active — activation blocked with a clear dependency message, consistent with "fail closed, not open." Module deactivated directly through Org Management (bypassing Marketplace) — catalog reflects current state correctly regardless of entry point. A listing is unpublished/retired while some orgs still have it active — existing activations remain fully functional; only new activations are blocked. (Future) third-party Extension submitted with an unsupported dependency — rejected at review, never silently accepted.

9. Business Rules

Marketplace never owns or duplicates activation logic, pricing logic, or module state — it is a discovery/presentation layer over Org Management's Module Registry and Billing's subscription state. Marketplace never owns or duplicates the Dependency Matrix. (Future) third-party Extensions are a distinct entity from first-party Business Modules and never receive equivalent trust by default. Catalog listings are never hard-deleted — retired/unpublished status only.

10. Validation Rules

A listing requires: name, category, description, and a reference to its actual module/pack identifier in the underlying registry. A listing cannot be published without valid dependency references that exist in the platform's Dependency Matrix.

11. Security Considerations

Standard RLS tenant isolation applies to any org-specific Marketplace data; catalog listings themselves are platform-wide, not tenant-

scoped. All catalog publish/unpublish/approve actions audit-logged. (Future) third-party Extensions explicitly flagged as requiring a dedicated security review/sandboxing model before implementation.

12. Multi-Tenant Considerations

The catalog itself is platform-wide (not org-scoped); an org's activation state against catalog listings remains strictly org-scoped, owned by Org Management/Billing as it already is today.

13. Subscription Considerations

Marketplace itself is not a subscribable module (like Notifications/Audit Logging/File Storage/Platform AI Layer, it's a platform-facing capability) — but everything it surfaces respects each target module's own existing subscription gating; Marketplace adds no new subscription behavior.

14. Notifications

New module/pack published to catalog → Owner/Admin (optional, low-priority) • (Future) Extension approved/rejected → submitting third-party developer.

15. Reports

Not applicable in v1 — catalog browsing/activation-conversion analytics deferred to future Analytics/Reports if ever needed.

16. Future Scalability

Full third-party developer program: submission, review, sandboxing, revenue share • extension permission model • ratings/reviews on listings • bundled "starter pack" listings • in-catalog upgrade/comparison flows tied into future pricing-model work.

17. Dependencies

Organization Management (Module Registry & Activation Model — Marketplace surfaces, never replaces) • Subscription & Billing Engine (activation gating, pricing display) • RBAC • Audit Logging • every business module (as catalog content, read-only reference to their metadata/dependencies).

18. Success Criteria

Every module in this Blueprint is browsable/activatable through one consistent storefront experience • no new activation authority or logic invented — Marketplace strictly presents and invokes what Org Management/Billing already own • dependency information shown before activation always accurate, sourced from the single locked Dependency Matrix • retiring a listing never breaks or deletes any org's existing active usage • the third-party Extension concept clearly scoped as distinct, not-yet-built.

Locked Decisions:

Third-Party Extensions: No third-party extension execution in Version 1. v1 only establishes the Marketplace architecture, extension entity, metadata model, and review lifecycle. Third-party code execution, sandboxing, permission isolation, API scopes, extension lifecycle management, revenue sharing, and developer tooling all remain future capabilities.

Marketplace Subscription Model: Marketplace is not a subscribable module. It is a core platform capability responsible only for module discovery, browsing, dependency visualization, and activation entry point. Subscription ownership, pricing, entitlement checks, and activation logic remain exclusively within the Subscription & Billing Engine and Organization Management.

Additional Locked Architectural Decisions

Immutable internal IDs on every Marketplace Listing, Extension, Category, Bundle, Review Request, Publication Record. Marketplace remains a metadata-driven catalog, never duplicates module configuration or business logic. Module dependency validation always references the centralized Dependency Matrix. Future extension APIs use versioned contracts. Future extension permissions follow the principle of least privilege and require explicit approval before accessing platform data. Marketplace exposes stable interfaces for future developer tooling without exposing internal platform implementation. Future licensing, billing, revenue sharing, extension analytics, ratings, reviews, digital signatures, and extension verification extend Marketplace without redesigning the core architecture. Marketplace supports future first-party and third-party extension packages through the same extension abstraction while maintaining separate trust and approval models.

12. Phase 6 - Cross-Cutting Platform Architecture

19 SECTIONS — LOCKED

The 17 module designs in Section 11 each answer "what does this module do." This phase answers the horizontal question that no single module owns: "how does every module behave the same way." These 19 sections apply platform-wide, exactly like Section 7's Global Architectural Principles, and were reviewed together — as a set — once all 17 modules were finalized, rather than being bolted on module-by-module. No module-level design in Section 11 is altered by this phase; these are additive, cross-cutting standards layered on top.

12.1 API Standards & Versioning

All module functionality is reachable through one REST, JSON, API-first surface (per System Architecture §11). URI-based versioning (`/api/v1/...`); additive, backward-compatible changes are preferred within a version, breaking changes require a new version path with a defined deprecation window for the old one. Resource naming, pagination (cursor-based for large collections), filtering, and sorting conventions are consistent across every module — no module invents its own pattern. An OpenAPI specification is generated from the implementation and kept in sync as the authoritative machine-readable contract feeding both internal tooling and the future Marketplace third-party Extension surface (Module 17).

12.2 Error Handling & Response Standards

A single standard error envelope (error code, human-readable message, correlation ID, field-level validation detail where applicable) is used platform-wide — no module defines its own error shape. HTTP status codes are used semantically: 400 for validation, 401/403 for authentication/permission, 404 for not found, 409 for conflict, 429 for rate-limited, 5xx for system failure. The correlation ID in every error response is the same identifier used in Logging & Observability (§12.8), letting a Super Admin trace a single failed request across logs, Audit Logging, and (if applicable) a Notification Framework delivery failure.

12.3 Rate Limiting & Throttling

Per-organization and per-user rate limits are enforced at the API layer to protect shared infrastructure — consistent with the Free-Tier First Design principle, since a single noisy tenant must never degrade Supabase's shared resources for every other tenant. Limits are configurable per endpoint sensitivity (authentication endpoints stricter than read endpoints, consistent with Module 1a's brute-force protections). A 429 response always carries a `Retry-After` value. Rate-limit violations are never silently dropped — they are logged for Super Admin visibility, consistent with the platform's audit-everything posture, though they are an operational log (§12.8), not a business Audit Event (Module 4).

12.4 Caching Strategy

Only read-heavy, infrequently-changing data is cached (org profile, module catalog listings, permission templates) — with a short TTL and an explicit invalidation hook tied to the owning module's write path. No cache may ever be trusted as an isolation boundary — RLS remains the enforced source of truth on every read, cache or not. Per Module 1c's locked permission-propagation NFR, no RBAC allow/deny decision is cached beyond the existing 60-second maximum propagation window.

12.5 Internationalization & Localization (i18n/l10n)

Every user-facing string is externalized to translation resource files from the first implementation, even though v1 ships a single locale — this realizes the BRD's "multi-language readiness built in from the start" NFR without building translation UI in v1. This is the same discipline already locked into the Notification Framework's versionable, localizable-ready template system (Module 3); this section extends that pattern to every module's own UI layer, not just notification content.

12.6 Multi-Currency & Multi-Timezone Handling

All timestamps are stored UTC-internal platform-wide, with timezone conversion happening exclusively in the presentation layer — this was already locked per-module in Attendance and Audit Logging; this section makes it a platform-wide, non-module-specific rule. All monetary values are stored with an explicit currency code and configurable precision/rounding (already locked in Payroll and Finance); an organization's configured default currency is a display/default concern only and never forces any consuming module to hardcode an assumption about it.

12.7 Data Validation & Input Sanitization Standards

A shared validation library is used by every module's forms and API endpoints rather than each module reimplementing its own. Server-side validation is always authoritative; any client-side validation is a UX convenience only and is never trusted as an enforcement boundary. Every value is sanitized against injection before persistence — with particular discipline around any field that is later rendered inside a Notification Framework template (Module 3's existing "never allow unescaped injection of untrusted content" rule), a Help Desk ticket, or a Projects/CRM comment field.

12.8 Logging & Observability (Monitoring, Metrics, Tracing)

Distinct from Audit Logging (Module 4), which is an immutable business-event record, not an operational one. Application logs, metrics, and traces are an operational concern, addressed free-tier-first (an open-source stack such as Grafana/Prometheus, or an equivalent free tier), correlated end-to-end via the same Correlation ID used in Error Handling (§12.2). Operational logs never store more PII than is strictly necessary for debugging — business-relevant PII belongs in Audit Logging, not operational logs.

12.9 Performance & Scalability Standards

Each endpoint class (read, write, report-generation) carries a target response-time budget defined at module-design time, not retrofitted after the fact. Database indexing strategy is reviewed per module as part of its design, particularly for any module with an immutable-ledger pattern (Attendance, Leave balance transactions, Inventory movements, Finance valuation entries) where read-time aggregation cost compounds over time. This section is the practical discipline that keeps System Architecture §10's "RLS + shared schema scales to thousands of organizations" claim true in production, not just in principle.

12.10 Backup & Disaster Recovery

Automated Supabase Postgres backups at a free-tier-appropriate cadence, with documented (not yet fully automated in v1) RPO/RTO targets. Audit Logs, Payroll runs/payslips, and Finance records are treated as the highest-priority recovery data given their compliance and financial sensitivity. Recovery drills are a documented manual process for v1; automated drill tooling is noted under Future Roadmap (Section 18).

12.11 Environment & Configuration Management

Distinct, fully isolated development, staging, and production environments, with no credential ever shared across them. Every external-provider credential (Razorpay, Resend, Supabase, and any future AI provider adapter selected under Module 16) lives in environment-scoped secrets, never hardcoded and never committed to source control — a direct extension of the Global Architectural Principle of Vendor Independence into the deployment layer.

12.12 CI/CD & Deployment Pipeline

GitHub (per Setup Instructions, Section 2) is the CI/CD substrate. Every module's Antigravity-implemented code passes automated build and test before merge — there is no direct-to-production path. Deployment remains module-aware in terms of what changed, but ships as part of the single platform surface (frontend + backend); a microservices split is explicitly not pursued in v1, consistent with Free-Tier First Design and Cost Philosophy.

12.13 Testing Strategy

Unit tests are mandatory (not optional) for business-rule-dense logic — RBAC's deny-wins resolution, Payroll's calculation engine, Leave Management's balance-transaction ledger — because this Blueprint's value is concentrated precisely in getting those business rules exactly right. Integration tests cover the cross-module contracts already locked in Section 11 (Leave's "On Leave" event into Attendance; Attendance/Leave's finalized snapshot into Payroll; Inventory's movement ledger into Finance). End-to-end tests cover the core commercial journey: signup → trial → module activation → first business action in each MVP module.

12.14 Security Hardening & Secrets Management

Consolidates the security posture already locked module-by-module (RLS primary/RBAC secondary, fail-closed by default, audit-everything) into one platform-wide checklist: dependency vulnerability scanning, zero secrets in source control, and signed-webhook verification (already locked for Billing's payment gateway in Module 2) applied as the standard pattern for any future inbound webhook from any provider adapter, not a Billing-specific exception.

12.15 Data Privacy & Compliance Readiness

The architecture keeps the door open for GDPR/SOC 2/ISO 27001 (per BRD §6) without gating MVP delivery on any of them, consistent with the BRD's original framing. A data-subject access/export/delete request necessarily spans multiple modules (Authentication's User, Employee Management's Employee record, CRM's Contact) rather than being owned by a single one — this cross-module capability is explicitly scoped as a future compliance initiative (Section 18), not built in v1.

12.16 Accessibility (a11y) Standards

WCAG 2.1 AA is the target standard for the React/Tailwind frontend across every module's UI, adopted from the first implementation rather than retrofitted module-by-module later.

12.17 Mobile Responsiveness & Cross-Browser Support

Mobile-responsive web across every module's UI, consistent with the BRD's non-functional requirements. Native mobile applications are explicitly out of scope for v1 and noted under Future Roadmap (Section 18).

12.18 Documentation Standards

Each module's approved 18-point design (Section 11) remains the primary functional documentation of record. This section adds the requirement for generated API documentation (the OpenAPI spec from §12.1) and a developer-facing README per module, both kept in sync as modules are implemented — required reading for the future third-party Extension developers Module 17's architecture anticipates.

12.19 Incident Response & On-Call Process

A lightweight, startup-appropriate incident process for v1: defined severity levels, the Platform Super Admin as default incident owner, and a post-incident write-up that feeds back into Audit Logging for any affected organization's trail. A formal on-call rotation is deferred until team size justifies it, noted under Future Roadmap (Section 18) rather than built prematurely.

13. Architectural Consistency Review & Applied Corrections

Once all 17 modules had individually completed their 18-point designs, a single consistency pass was run across the whole Blueprint — checking for exactly the kind of drift that module-by-module design (each finalized independently, in sequence, without revisiting earlier modules) can quietly introduce. Three corrections were identified and applied. Each is documented here for traceability, in addition to being called out inline at its point of application.

Correction 1 — Module Classification: Modules 16 & 17 Reclassified

Reviewed: Section 6 (Phase 4 – Module Planning & Build Sequence), where Modules 16 ("AI Assistant") and 17 ("Marketplace") were originally scoped as *Business Module (Roadmap)* — the same customer-subscribable category as CRM, Projects, and Help Desk.

Issue found: Once their 18-point designs were finalized, both turned out to be architecturally non-subscribable, platform-wide shared capabilities consumed by other modules on an org's behalf — not something an Owner independently subscribes to the way they would CRM or Payroll. Leaving them classified as ordinary Business Modules would have implied they sit behind Billing's per-module subscription toggle (Module 2), which neither design actually supports.

Correction applied: Module 16 renamed "AI Assistant" → "Platform AI Layer" and Module 17 "Marketplace" retained its name; both recategorized from *Business Module (Roadmap)* to a new *Shared Platform Capability* category, joining Notifications/Audit Logging/File Storage as internal, non-subscribable platform services.

Affected: Section 6 (Phase 4 table), Section 3 (Phase 1 Key Locked Decisions), Section 5.3 (System Architecture — Platform Services Layer), Section 8.1 (High-Level Architecture — new Shared Platform Capability Layer row), Sections 11.18 and 11.19 (module tags and Subscription Considerations points).

Correction 2 — Entity Ownership Completeness: Ticket and AI Layer Non-Duplication Rules

Reviewed: Section 9 (Entity Ownership Summary) and Section 10 (Module Dependency Matrix), extended to cover Modules 12–17.

Issue found: Two gaps surfaced. (a) Help Desk's Ticket entity had no explicit "never merges with Project Task" callout, despite Module 11's Projects design already locking the reverse statement — an asymmetry that could let a future designer conflate the two entities from the Help Desk side. (b) The Platform AI Layer's entity row had no explicit non-persistence rule, creating a risk that it could silently become a second, ungoverned store of other modules' content — directly against the "Never a second source of truth" pattern already locked for Analytics and Marketplace.

Correction applied: Ticket's entity row extended with "Ticket never merges with Project Task," matching Module 11's existing statement from the other direction. The AI Request/Capability entity row extended with "AI Layer never persists calling-module content, only metering metadata." "Never a second source of truth" was promoted from an Analytics/Marketplace-specific note into a platform-wide Recurring Pattern in Section 7, now explicitly naming the Platform AI Layer alongside Analytics and Marketplace.

Affected: Section 9 (Entity Ownership Summary rows for Ticket and AI Request/Capability), Section 7 (Recurring Platform-Wide Patterns), Section 11.14 and 11.18 (Business Rules points).

Correction 3 — Module Registry & Activation Ownership Confirmed

Reviewed: Section 5.6 (System Architecture — Module Registry & Activation Model), in light of Module 17's finalized Marketplace design.

Issue found: Three modules each touched "which modules are active for an organization" — Organization Management (structural presence, since Module 1b), Subscription & Billing (entitlement/gating, since Module 2), and now Marketplace (discovery/browsing, Module 17). Without one explicit owner, this was on a path toward exactly the "parallel logic" anti-pattern the platform has deliberately avoided everywhere else (Section 7).

Correction applied: Module Registry & Activation Model ownership confirmed exclusively as Organization Management. Subscription & Billing continues to own entitlement/gating only — it reads the registry, it does not own it. Marketplace is confirmed as a strictly read/invoke discovery layer with no independent registry state of its own.

Affected: Section 5.6, Section 9 (Entity Ownership Summary — Module Registry & Activation State row), Module 1b's Features (Section 11.2), Module 17's Business Rules and Locked Decisions (Section 11.19).

No correction altered any module's core business rules, permissions, or workflows — all three were classification, ownership-attribution, and cross-reference-completeness corrections, found and closed before any Antigravity implementation prompt is generated for the affected modules.

14. Appendix – Working Process Reference

The Phased Workflow

Product Vision → Business Requirements → System Architecture → Module Planning → Build Module Design Completely (18 points) → Get Explicit Approval → Generate Antigravity Prompt → Implement → Review → Fix → Approve → Next Module. As of this export, every module (1a–17) has completed the design-and-approval portion of this workflow; the Antigravity-prompt-onward portion remains a separate, per-module deliverable not included here.

Every Module Design Covers

Purpose • Business Problems Solved • Users Involved • User Roles • Permissions • Features • Workflows • Edge Cases • Business Rules • Validation Rules • Security Considerations • Multi-Tenant Considerations • Subscription Considerations • Notifications • Reports • Future Scalability • Dependencies on Other Modules • Success Criteria

Rules That Never Change

- No implementation prompt is generated until all 18 points are finalized and approved.
- One module at a time, in sequence — no skipping ahead, no mixing designs or prompts from different modules.
- External dependencies (API keys, payment gateways, SMS/AI providers, etc.) are never assumed — always surfaced explicitly for a decision before design proceeds. **The Platform AI Layer's provider decision (Module 16) remains deliberately open as a provider-agnostic stub — no default provider is locked, pending an explicit future decision.**
- Every implementation prompt is self-contained and never assumes context from earlier conversation.
- Global architectural principles (Section 7) and the Cross-Cutting Platform Architecture (Section 12) apply to every module without needing to be restated.
- Architectural consistency reviews (Section 13) are run whenever a batch of modules is finalized together, not only at the very end — this export's review is the first full-platform pass, now that all 17 modules exist.

What Comes Next

With all 17 modules and the Cross-Cutting Platform Architecture finalized, the next step on request is generating the first Antigravity implementation prompt — starting from Module 1a, per the locked build sequence — one module at a time, exactly as this Appendix has always specified.

15. Glossary

Term	Definition
Tenant / Organization	A fully isolated customer entity; data, membership, roles, permissions, and subscription state never cross tenant boundaries (BRD §1).
RLS (Row-Level Security)	PostgreSQL feature enforcing tenant isolation at the database layer; the platform's primary isolation boundary (System Architecture §1).
RBAC	Role-Based Access Control; the fine-grained, configurable permission layer on top of RLS (Module 1c). Never a substitute for RLS.
ABAC	Attribute-Based Access Control; an explicitly deferred future extension beyond v1's RBAC (BRD §2, Module 1c §16).
Membership	The link between a User, an Organization, and a coarse platform Role; one User may hold many Memberships (System Architecture §2).
Structure Node	A unified, hierarchical org-structure entity (Branch/Department/Team/Business Unit) owned by Organization Management (Module 1b).
Core Directory	The set of platform-owned, shared entities (Structure Node, Employee, Contact/Account, Item/Location/Movement) that business modules reference but never fork (System Architecture §4).
Module Registry & Activation Model	The record of which modules are active for an organization; owned exclusively by Organization Management (System Architecture §6; Correction 3).
Trial / Grace Period / Locked / Reactivated	The four states of the Subscription enforcement state machine owned by the Subscription & Billing Engine (Module 2).
Read-only-during-Locked	The standard pattern where a Locked organization can view but not modify data in a given module (Section 7).
Provider Abstraction	An internal interface behind which any external service (payment, email, storage, AI, etc.) sits, so the provider can be swapped without redesigning business logic (Global Architectural Principles, Section 7).
Immutable Ledger	A pattern used by Attendance, Leave balances, Inventory Stock Movements, and Finance Valuation Entries, where the current value is always derived from an append-only transaction history, never a directly editable field.
Fail Closed	The rule that any unrecognized or unconfigured access/notification/AI-capability check defaults to denial, never silent success (Section 7).
Never a Second Source of Truth	The rule that Analytics, Marketplace, and the Platform AI Layer read through owning modules' stable interfaces rather than becoming secondary data stores (Section 7; Correction 2).
Correlation ID	A unique identifier tying a single request across API error responses, operational logs, and (where applicable) Audit Events (Cross-Cutting §12.2, §12.8).
Shared Platform Capability	The category (introduced in Correction 1) for internal, non-subscribable platform services consumed by every module — includes Notifications, Audit Logging, File Storage, the Platform AI Layer, and Marketplace.
Finalized Run	An immutable Payroll Run state; post-finalization corrections are always a new adjustment transaction, never an edit to history (Module 9).
Extension	A conceptual, not-yet-implemented third-party-built unit distinct from a first-party Module, scoped architecturally in Module 17 but not executable in v1.

16. Decision Log

A consolidated index of the Blueprint's locked decisions and where each was made, for quick cross-reference without re-reading every module in full.

Decision	Locked In	Primarily Affects
Razorpay as v1 payment provider, behind a Payment Provider abstraction	Section 2; Module 2	Billing
Resend as v1 email provider, behind a Notification Provider abstraction	Section 2; Module 3	Notifications
Platform AI Layer provider: deliberately provider-agnostic stub, no default selected	Section 2; Module 16	Platform AI Layer (open decision — see Section 18)
Help Desk v1 includes external self-service requester login	Module 12	Help Desk
Help Desk v1 includes basic SLA timers and simple escalation	Module 12	Help Desk
14-day trial → 7-day grace period → Locked, all automatic	BRD §3; Module 2	Billing, every subscribable module
Locked ≠ Cancelled; only explicit Cancel starts the 30-day retention/deletion clock	BRD §4; Module 2	Billing, Organization Management
Deny-wins on conflicting RBAC grants; fail closed on unknown combinations	Module 1c	RBAC, every module's access checks
No hard deletes anywhere — status/lifecycle transitions and archiving only	Section 7	All modules
Immutable internal IDs on every core entity, platform-wide	Section 7	All modules
Whole-day leave only in v1; half-day/hourly deferred	Module 8	Leave Management
A Finalized Payroll Run is immutable; corrections are new adjustment transactions	Module 9	Payroll
Contact and Account are shared, platform-level Core Directory entities	Module 10	CRM, Projects, Help Desk, Finance
Project Task and Help Desk Ticket remain permanently distinct entities	Module 11; Module 12; Correction 2	Projects, Help Desk
Stock level is never directly editable — always derived from the movement ledger	Module 13	Inventory, Finance
No third-party Extension code execution in v1 — architecture/metadata only	Module 17	Marketplace
Modules 16 & 17 reclassified as Shared Platform Capability, not Business Module	Correction 1	Platform AI Layer, Marketplace
Module Registry & Activation Model owned exclusively by Organization Management	Correction 3	Organization Management, Billing, Marketplace
UTC-internal timestamps everywhere; timezone conversion only in presentation layer	Cross-Cutting §12.6; Section 7	All modules
WCAG 2.1 AA as the accessibility target from v1	Cross-Cutting §12.16	All modules (frontend)

17. Version History

Version	Scope	Notes
v1	Module 1a (Authentication) only	Initial scaffold establishing Phases 1–4 and the first complete 18-point module design.
v2	Modules 1a–11 finalized (Platform Foundation complete + MVP suite through Payroll + CRM, Projects)	Exported as PDF. Confirmed MVP launchable cutoff at Module 9. Modules 12–17 explicitly scoped as not-yet-designed roadmap items, per that export's own locked rule against skipping ahead.
v3 (this document)	Modules 1a–17 fully finalized, plus Phase 6 Cross-Cutting Platform Architecture (19 sections) and a full-platform Architectural Consistency Review (3 corrections applied)	Supersedes v1 and v2. No Antigravity implementation prompts included. Every module in the original roadmap now has a complete, approved design.

Note: v2 and v3 were generated on the same date in this workspace; version numbering reflects design-completeness milestones (checkpoints), not calendar time.

18. Future Roadmap Beyond Module 17

With all 17 originally-scoped modules finalized, "future roadmap" now means capabilities explicitly deferred *within* already-locked designs, consolidated here from each module's own Future Scalability point rather than invented fresh:

- **AI provider decision (Module 16):** still explicitly open — provider-agnostic stub only, no default locked. The next module-level decision that needs to be explicitly raised, per the platform's "external dependencies never assumed" rule.
- **Third-party Extension execution (Module 17):** full developer program — submission, review, sandboxing, permission model, revenue share. v1 ships architecture and metadata only.
- **ABAC** (Module 1c) and **multi-step, role-based approval chains** (Leave Management, Module 8) as richer access/workflow models beyond v1's RBAC + single-level approval.
- **OAuth/SSO/SAML** (Module 1a) for enterprise-grade identity federation beyond email+password/MFA.
- **Regional statutory Compliance Packs** (Payroll, Module 9; Finance, Module 14) for India/UAE/Saudi Arabia/Singapore and beyond, extending — never modifying — each module's core calculation engine.
- **Direct payroll disbursement / bank integration** (Module 9), currently calculate-and-record only.
- **Full GDPR / SOC 2 / ISO 27001 compliance program** (BRD §6; Cross-Cutting §12.15), architecturally anticipated but not built to certification-readiness in v1.
- **Database-per-tenant migration path** (System Architecture §10) for select enterprise customers outgrowing shared-schema RLS.
- **Natural-language analytics querying** (Analytics, Module 15) via the Platform AI Layer's stable capability interface, once an AI provider is selected.
- **Native mobile applications** (Cross-Cutting §12.17), beyond v1's mobile-responsive web.
- **Formal on-call rotation and automated disaster-recovery drills** (Cross-Cutting §12.19, §12.10), deferred until team size and customer scale justify them.
- **Cross-module data-subject request handling** (Cross-Cutting §12.15) as a dedicated compliance-initiative capability spanning Authentication, Employee Management, and CRM.

Consistent with the Blueprint's own rule (Section 14): none of the above are designed here. Each becomes a full 18-point module design, one at a time, only when explicitly picked up.

19. Final Architecture Summary

The platform is a single, layered system: a React/Vite/Tailwind client sits above a Platform Foundation layer (Authentication, Organization Management, RBAC, Subscription & Billing, Notification Framework, Audit Logging, File Storage), a Shared Platform Capability layer (Platform AI Layer, Marketplace — Correction 1), and a Business Modules layer (Employee Management, Attendance, Leave Management, Payroll, CRM, Projects, Help Desk, Inventory, Finance, Analytics & Reports). Every layer is reachable through one API-first surface (Cross-Cutting §12.1).

Tenant isolation is enforced twice, deliberately: PostgreSQL RLS as the non-negotiable primary boundary, and RBAC as a finer-grained secondary layer that never substitutes for it. Every external dependency — payment, email, storage, and (when eventually selected) AI — sits behind a provider abstraction, so vendor swaps never require business-logic redesign. Every sensitive action is audit-logged, append-only, immutable. Every core entity carries a stable, immutable ID. Nothing is ever hard-deleted. Every business module is gated by a single, centralized Billing enforcement point rather than reimplementing its own subscription check. And, as confirmed by this export's consistency review, no module ever becomes a second, ungoverned source of truth for another module's data — Analytics, Marketplace, and the Platform AI Layer are all read-through, presentation/discovery/capability layers over the modules that actually own their data.

All 17 modules from the original roadmap (Section 6) are now finalized and approved at the design level. The Cross-Cutting Platform Architecture (Section 12) locks in the horizontal standards — API, error handling, i18n, observability, security, testing, and more — that apply identically across all of them. Three consistency corrections (Section 13) were found and closed in a full-platform review, none altering any module's core business rules. Per the Blueprint's own unchanging rule, no Antigravity implementation prompt has been generated for any module in this export — that remains a deliberate, separate, one-module-at-a-time next step, starting from Module 1a.